

**LAPORAN UJIAN TENGAH SEMESTER
MATA KULIAH DATA MINING**



DOSEN PENGAMPU

Dr. Eng. I Putu Agung Bayupati, S.T.,M.T.

DISUSUN OLEH:

Dewa Putu Ananta Prayoga	(2305551007)
Anak Agung Made Krishna Mahendrayana	(2305551018)
I Putu Arya Putra Raditya	(2305551042)

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS UDAYANA

BALI

2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iv
DAFTAR KODE PROGRAM	v
DAFTAR TABEL.....	6
BAB I PENDAHULUAN.....	7
1.1 Latar Belakang	7
1.2 Rumusan Masalah	8
1.3 Tujuan Penulisan.....	8
1.4 Manfaat Penulisan.....	8
1.5 Batasan Masalah.....	9
BAB II TINJAUAN PUSTAKA	10
2.1 Data Mining	10
2.2 Forecasting	11
2.3 Association.....	11
2.4 Classification.....	12
2.5 Clustering	13
2.6 Estimation	14
2.7 Apriori.....	14
2.8 K-Means.....	15
2.9 Logistic Regression.....	15
BAB III METODE PENELITIAN.....	17
3.1 Algoritma Apriori.....	17
3.2 K-Means.....	18
3.3 Logistic Regression.....	18

3.3.1	Prapemrosesan dan Normalisasi Data (Z-Score Standardisation)	18
3.3.2	Perhitungan Kombinasi Linear (Forward Pass)	19
3.3.3	Transformasi Non-linear (Aktivasi Sigmoid)	19
3.3.4	Evaluasi Tingkat Kesalahan (Cost Function).....	19
3.3.5	Perhitungan Turunan Parsial (Backward Pass)	19
3.3.6	Pembaruan Parameter (Gradient Descent Update)	20
3.3.7	Pengambilan Keputusan (Thresholding) dan Evaluasi Akhir	20
BAB IV HASIL DAN PEMBAHASAN.....		21
4.1	Algoritma Apriori.....	21
4.1.1	F2	22
4.1.2	F3	25
4.2	K-Means.....	27
4.2.1	Kode Program	28
4.2.2	Hasil	32
4.3	Logistic Regression.....	36
4.3.1	Perhitungan Manual	38
4.3.2	Kode Program	48
4.3.3	Hasil	55
BAB V PENUTUP.....		56
5.1	Kesimpulan	56
5.2	Saran.....	57
DAFTAR PUSTAKA		58

DAFTAR GAMBAR

Gambar 4.1 Gambar Hasil Program Menghitung F2	24
Gambar 4.2 Hasil Menghitung F3.....	27
Gambar 4.3 Hasil <i>Clustering</i> menggunakan K-Means	35
Gambar 4.4 Gambar Hasil Evaluasi Model	55

DAFTAR KODE PROGRAM

Kode Program 4.1 Kode Program Persiapan Data	21
Kode Program 4.2 Kode Program Menghitung F2	23
Kode Program 4.3 Kode Program Menghitung F3	26
Kode Program 4.4 <i>Clustering</i> menggunakan K-Means	31
Kode Program 4.5 Persiapan <i>Dataset</i>	48
Kode Program 4.6 Rumus 1: Persamaan Linear	50
Kode Program 4.7 Rumus 2: Aktivasi <i>Sigmoid</i>	50
Kode Program 4.8 Rumus 3: <i>Cost Function (Log Loss)</i>	50
Kode Program 4.9 Rumus 4: <i>Gradient Descent</i>	51
Kode Program 4.10 Proses <i>Training</i> dan Evaluasi Model	53

DAFTAR TABEL

Tabel 4.1 Data <i>Clustering</i> menggunakan K-Means	28
Tabel 4.2 Data Pasien	37
Tabel 4.3 Hasil Standardisasi Data.....	38
Tabel 4.4 Hasil Nilai Persamaan Linear.....	39
Tabel 4.5 Hasil Nilai Probabilitas	40
Tabel 4.6 Hasil Nilai Persamaan Linear.....	42
Tabel 4.7 Hasil Nilai Probabilitas	43
Tabel 4.8 Hasil Nilai Persamaan Linear.....	44
Tabel 4.9 Hasil Nilai Probabilitas	45

BAB I

PENDAHULUAN

Bab I Pendahuluan ini membahas latar belakang, perumusan masalah, tujuan, manfaat, dan batasan masalah terkait implementasi algoritma data mining dan machine learning. Pembahasan difokuskan pada komputasi dan analisis pola data, mulai dari penemuan aturan asosiasi pada pola keranjang belanja menggunakan algoritma Apriori, segmentasi data tak berlabel (*clustering*) berdasarkan karakteristik usia dan pendapatan menggunakan algoritma K-Means, hingga pemodelan klasifikasi prediktif tingkat lanjut berbasis Logistic Regression di bidang medis untuk memprediksi probabilitas risiko penyakit jantung pada pasien.

1.1 Latar Belakang

Seiring dengan berkembangnya teknologi sistem basis data, *data mining* menjadi proses komputasi yang sangat penting untuk mencari, mengidentifikasi, dan mengekstraksi pola-pola tersembunyi yang berpotensi berguna dari dalam suatu basis data guna memprediksi probabilitas kejadian di masa depan. Berbagai teknik *data mining* dapat diimplementasikan untuk menyelesaikan ragam permasalahan di berbagai sektor, baik untuk pengenalan pola keranjang belanja (*market basket analysis*), penentuan segmentasi, maupun klasifikasi prediksi berbasis kelas.

Dalam laporan ini, dikaji tiga algoritma fundamental *data mining* beserta simulasi perhitungannya pada permasalahan yang spesifik. Pertama, algoritma Apriori digunakan untuk analisis asosiasi guna mengungkap keterkaitan pembelian kombinasi barang belanja (seperti susu, gula, teh, kopi, dan roti) pada data transaksi pelanggan. Kedua, algoritma K-Means diterapkan sebagai teknik *clustering* untuk mengidentifikasi pengelompokan alami masyarakat berdasarkan tingkat usia (*Age*) dan pendapatan (*Income*). Terakhir, algoritma Logistic Regression digunakan sebagai model klasifikasi untuk memprediksi risiko penyakit jantung pasien berdasarkan variabel medis seperti umur, status perokok, kolesterol, tekanan darah, dan diabetes. Melalui penerapan ketiga algoritma ini, diharapkan dapat dipahami komputasi matematis dan evaluasi pola dari masing-masing metode secara komprehensif.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penulisan laporan ini adalah:

1. Bagaimana proses pembentukan aturan asosiasi dan pencarian kombinasi barang (2-itemset dan 3-itemset) pada data transaksi keranjang belanja menggunakan algoritma Apriori
2. Bagaimana cara menentukan jumlah kluster yang optimal dan proses pengelompokan data observasi usia serta pendapatan menggunakan algoritma K-Means
3. Bagaimana alur komputasi matematis pemodelan klasifikasi biner menggunakan algoritma Logistic Regression dalam memprediksi risiko penyakit jantung pasien

1.3 Tujuan Penulisan

Tujuan dari penulisan laporan eksperimen *data mining* ini adalah:

1. Mengimplementasikan algoritma Apriori untuk membedah dan menghitung nilai probabilitas keterkaitan antar barang (support dan confidence) pada data transaksi.
2. Menerapkan metode clustering dengan algoritma K-Means untuk mengekstraksi segmentasi kelompok data Age dan Income berdasarkan kedekatan karakteristiknya.
3. Membangun dan memahami tahapan kalkulasi algoritma Logistic Regression, mulai dari forward propagation hingga backward propagation untuk memecahkan kasus prediksi risiko penyakit jantung.

1.4 Manfaat Penulisan

- Bagi Penulis/Mahasiswa: Memberikan fondasi teoritis dan teknis yang kuat, baik secara matematis maupun pemrograman (seperti penggunaan bahasa Python, pustaka Pandas, dan Scikit-Learn), terkait komputasi algoritma penambangan data, mulai dari pencarian pola keranjang belanja tingkat dasar (algoritma Apriori), pengelompokan data tak berlabel (K-

Means), hingga ekstraksi pemodelan klasifikasi prediktif (*Logistic Regression*) yang mencakup perhitungan propagasi maju dan mundur.

- Bagi Akademisi: Dapat digunakan sebagai referensi komprehensif atau literatur terapan implementasi fundamental algoritma *data mining* yang memadukan teori probabilitas bersyarat, metode kalkulasi statistik (seperti standardisasi *Z-Score* dan minimisasi metrik *Log Loss*), serta pembelajaran mesin kategori *supervised* dan *unsupervised learning*.
- Bagi Praktisi Terapan: Menyediakan wawasan analitik dan purwarupa (*prototype*) dasar solusi pengambilan keputusan berbasis data (*data-driven*), misalnya untuk merancang strategi tata letak barang ritel silang melalui *market basket analysis*, menentukan target kampanye komersial melalui segmentasi demografi pendapatan pelanggan, hingga menciptakan alat pendukung diagnosis klinis otomatis untuk memprediksi probabilitas risiko penyakit jantung pada sektor layanan medis.

1.5 Batasan Masalah

Agar pembahasan lebih terfokus, penulisan laporan ini memiliki batasan masalah sebagai berikut:

1. Implementasi algoritma Apriori dibatasi pada pencarian kombinasi maksimal himpunan tiga barang (F3) menggunakan 10 sampel data transaksi, dengan penetapan ambang batas minimum support sebesar 20% dan minimum confidence sebesar 35%.
2. Percobaan algoritma K-Means diterapkan pada 10 sampel data yang hanya terdiri dari dua variabel tak terstruktur (Age dan Income), dengan penentuan jumlah klaster optimal (k) menggunakan metode visualisasi Elbow.
3. Pemodelan algoritma Logistic Regression dibatasi pada studi kasus klasifikasi biner (dua kelas) menggunakan 20 sampel data pasien. Prosesnya mencakup standardisasi (*Z-Score*), fungsi aktivasi Sigmoid, serta evaluasi kesalahan berbasis *Log Loss*.

BAB II

TINJAUAN PUSTAKA

Bab II Tinjauan Pustaka ini memaparkan landasan teori yang mendasari algoritma dan metode penambangan data (*data mining*) yang digunakan di dalam laporan. Topik-topik yang dibahas mencakup pemahaman mendasar mengenai *Knowledge Discovery in Database* (KDD) dan eksplorasi *data mining*, konsep dasar pencarian aturan asosiasi (*association rule*) melalui analisis pola frekuensi tinggi menggunakan algoritma Apriori, metode pengelompokan data tanpa label (*clustering*) menggunakan algoritma K-Means, hingga pemodelan klasifikasi biner tingkat lanjut menggunakan algoritma *Logistic Regression*. Selain itu, dibahas pula teori pendukung seperti kalkulasi standardisasi data (Z-Score), fungsi aktivasi Sigmoid, evaluasi metrik *Log Loss*, serta landasan medis mengenai prediksi probabilitas risiko penyakit jantung pada pasien.

2.1 Data Mining

Data mining adalah sebuah proses komputasi interdisipliner yang erat kaitannya dengan *Knowledge Discovery in Database* (KDD), yang berevolusi sebagai respons atas keterbatasan pendekatan statistik tradisional dalam menangani *big data*. Pendekatan ini berfungsi untuk mencari, mengidentifikasi, dan mengekstraksi pola-pola tersembunyi serta korelasi yang berpotensi berguna di dalam suatu basis data berukuran masif guna memprediksi tren dan probabilitas kejadian di masa depan. Dalam penerapannya, *data mining* bekerja dengan menyatukan proses dialektis deduktif dan induktif melalui perpaduan tiga fondasi keilmuan utama, yaitu statistik klasik, kecerdasan buatan (*machine learning*), dan teknologi sistem basis data. Proses penggalian informasi ini sangat bergantung pada implementasi serangkaian teknik dan metode komprehensif, seperti klasifikasi, klusterisasi (*clustering*), deteksi anomali, analisis regresi, hingga pembelajaran aturan asosiasi (*association rule learning*). Keunggulan dari *data mining* adalah kemampuannya yang sangat tangguh untuk memproses kumpulan data bervolume raksasa baik yang terstruktur maupun tidak terstruktur, mampu bekerja secara otomatis atau semi-otomatis untuk menguraikan efek korelasi variabel yang sangat kompleks dan non-linear, serta secara signifikan dapat meningkatkan akurasi dari suatu model prediksi.

2.2 Forecasting

Forecasting (peramalan) adalah proses komputasi prediktif untuk memprakirakan kejadian, nilai pasti (*price prediction*), atau arah pergerakan tren (*trend prediction*) di masa depan guna membantu pembuat kebijakan dan analisis dalam mengoptimalkan strategi serta mengelola risiko. Mengingat kondisi masa depan yang selalu memiliki ketidakpastian dan parameter yang bersifat non-linear, sistem peramalan yang ideal tidak hanya menghasilkan satu skenario mutlak, melainkan memodelkan distribusi probabilitas untuk melihat rentang berbagai kemungkinan skenario yang bisa terjadi beserta tingkat risikonya. Secara metodologis, *forecasting* telah berevolusi dari pendekatan statistik tradisional (seperti model ARIMA) dan simulasi fisika (*Numerical Weather Prediction/NWP*) yang memiliki keterbatasan dalam menangani data non-linear dan kompleks, menuju pendekatan berbasis data (*data-driven*) dengan menggunakan arsitektur *machine learning* dan *deep learning*. Dalam penerapannya, *forecasting* modern bekerja dengan mengekstraksi fitur dan menemukan pola tersembunyi dari data historis secara otomatis menggunakan teknologi kecerdasan buatan tingkat lanjut, seperti *Recurrent Neural Networks* (RNN/LSTM/GRU) untuk menangkap dinamika temporal, model *Transformer* dan model Difusi untuk memproses ketergantungan data dalam rentang panjang secara paralel serta menghasilkan sampel peramalan probabilistik, hingga *Large Language Models* (LLM) untuk memproses data non-numerik seperti teks dan sentimen media. Keunggulan utama dari teknologi *forecasting* saat ini adalah sifat interdisipliner yang sanggup menggabungkan pembelajaran multimodal untuk memproses data berskala raksasa, sehingga mampu mengurai interaksi efek prediktor yang sangat kompleks dan secara signifikan menghasilkan prediksi masa depan yang lebih komprehensif, cepat, dan akurat di berbagai domain, mulai dari fluktuasi pasar saham hingga cuaca ekstrem.

2.3 Association

Aturan asosiasi (*Association Rule*) adalah salah satu metode utama dalam *data mining* yang digunakan untuk mencari dan menemukan pola kombinasi

(*itemset*) yang bermakna dan paling sering muncul secara bersamaan di dalam sekumpulan transaksi atau basis data yang besar. Teknik yang sering juga disebut sebagai *affinity analysis* atau *market basket analysis* ini bekerja dengan cara memeriksa semua kemungkinan hubungan sebab-akibat (pola *if-then*) antar berbagai item, lalu memilih indikator yang paling mungkin (*most likely*) untuk menunjukkan adanya hubungan ketergantungan antar item tersebut. Dalam proses analitiknya, pencarian asosiasi ini selalu bergantung pada dua tolok ukur utama, yaitu nilai *support* dan nilai *confidence*. *Support* (nilai penunjang) berfungsi untuk mengukur persentase seberapa sering suatu kombinasi item muncul di dalam seluruh data transaksi, sedangkan *confidence* (nilai kepastian) digunakan untuk mengukur seberapa kuat hubungan keterkaitan antara dua item tersebut pada kondisi tertentu. Pada akhirnya, tujuan dari analisis asosiasi adalah untuk menemukan semua aturan valid yang berhasil memenuhi syarat ambang batas *minimum support* dan *minimum confidence*, yang kemudian hasil ekstraksi aturan ini dapat digunakan untuk menemukan wawasan tersembunyi yang berharga, seperti mengenali pola kombinasi barang yang dibeli konsumen atau menemukan pola karakteristik hubungan pada data kelulusan mahasiswa (*tracer study*).

2.4 Classification

Klasifikasi dalam *data mining* adalah sebuah proses untuk menemukan model prediktif yang membagi sekumpulan data ke dalam kelompok-kelompok atau kelas tertentu berdasarkan batasan yang telah ditentukan. Berbeda dengan metode estimasi atau regresi yang memprediksi arah nilai numerik, klasifikasi beroperasi secara spesifik menggunakan target variabel yang bersifat kategori (data yang terputus-putus dan tidak teratur). Secara historis, konsep klasifikasi ini pada awalnya diadaptasi dari bidang biologi oleh Carolus Linnaeus yang pertama kali mengelompokkan spesies tanaman berdasarkan karakteristik fisiknya. Dalam konteks komputasi modern dan dunia bisnis, klasifikasi memegang peranan krusial untuk memecahkan berbagai kasus prediktif berbasis kelas, seperti mengidentifikasi apakah suatu transaksi kartu kredit tergolong penipuan atau bukan, memperkirakan kelayakan pengajuan kredit nasabah (kredit baik atau buruk), hingga mendiagnosis jenis penyakit seorang pasien. Untuk menjalankan

tugas ini, terdapat berbagai macam algoritma pembelajaran mesin (*machine learning*) yang bisa diimplementasikan, di antaranya adalah metode Pohon Keputusan (*Decision Tree* seperti algoritma C4.5 dan CART), *Support Vector Machine* (SVM), *K-Nearest Neighbour* (KNN), Jaringan Bayesian, hingga Algoritma Genetik. Selanjutnya, model klasifikasi yang telah dibangun menggunakan algoritma tersebut dapat dievaluasi tingkat akurasi menggunakan metode pengujian tertentu, seperti metode *hold-out* atau validasi silang (*cross-validation*).

2.5 Clustering

Klastering (*clustering*) adalah sebuah teknik analisis data di dalam *data mining* dan cabang utama dari pembelajaran mesin tanpa pengawasan (*unsupervised learning*) yang berfungsi untuk mengelompokkan objek-objek data yang tidak memiliki label (*unlabeled data*) dengan panduan manusia yang sangat minim atau bahkan tidak ada sama sekali. Tujuan utama dari metode ini adalah untuk mengungkapkan struktur bawaan atau kelas-kelas tersembunyi di dalam suatu himpunan data, di mana hubungan antar objeknya belum diketahui secara pasti sebelumnya. Dalam praktiknya, algoritma klastering bekerja dengan cara membagi kumpulan data ke dalam kelompok-kelompok terpisah sedemikian rupa sehingga objek-objek yang tergabung dalam satu kelompok (*cluster*) yang sama akan memiliki kesamaan karakteristik yang tinggi, sementara objek-objek yang berada di kelompok lain akan memiliki sifat yang berbeda. Berbeda dengan tugas klasifikasi yang sudah memiliki variabel target atau kelas yang pasti, klastering murni mengklasifikasikan observasi data atau variabel independen secara mandiri untuk berbagai keperluan awal analitik, seperti mengenali pola, merangkum data, mendeteksi anomali (pencilan), dan mengurangi jumlah dimensi data (*dimensionality reduction*). Karena tidak ada satu pun algoritma klastering yang bisa berlaku universal untuk semua kondisi, metode ini diimplementasikan melalui berbagai ragam pendekatan yang disesuaikan dengan jenis dan volume data, di antaranya mencakup metode berbasis jarak (*distance-based* seperti K-Means), metode hierarkis, berbasis kepadatan (*density-based*), hingga berbasis grid. Pada akhirnya, teknik pengelompokan tingkat lanjut ini menjadi instrumen esensial

untuk mengekstraksi wawasan berharga dari himpunan data berdimensi tinggi di berbagai sektor, mulai dari segmentasi pasar, layanan kesehatan, hingga analisis jaringan sosial.

2.6 Estimation

Estimasi dalam *data mining* adalah sebuah proses analitik prediktif yang berfungsi untuk menghitung, memperkirakan, atau menentukan nilai numerik (angka riil) secara presisi dari suatu variabel target (variabel dependen/akibat) berdasarkan pola hubungannya dengan satu atau lebih variabel prediktor (variabel independen/penyebab). Berbeda dengan tugas klasifikasi yang memprediksi label kategori, proses estimasi murni berfokus pada prediksi nilai kontinu dengan cara mengekstraksi dan mengenali pola-pola ketergantungan yang penting dari dalam basis data historis. Dalam penerapannya, metodologi estimasi membentang dari pendekatan statistik tradisional seperti metode Regresi Linier Berganda yang menguji sejauh mana hubungan sebab-akibat linier antarvariabel, hingga implementasi algoritma *machine learning* dan *deep learning* tingkat lanjut seperti *Artificial Neural Network* (ANN), *Support Vector Machine* (SVM) untuk tugas regresi, dan metode *Ensemble* (*Random Forest*, *XGBoost*) yang sangat tangguh dalam menangani himpunan data berdimensi tinggi serta mengurai dinamika parameter yang non-linear. Pada praktiknya, kemampuan model estimasi ini memegang peranan krusial sebagai landasan pengambilan keputusan strategis, optimalisasi sistem, dan mitigasi risiko kerugian di berbagai sektor kehidupan. Sebagai contoh nyata, teknik ini diandalkan dalam ranah lingkungan dan energi untuk mengestimasi kalkulasi volume emisi karbon saat ini dan di masa depan secara akurat guna mendukung kebijakan iklim, serta diaplikasikan dalam sektor industri untuk mengestimasi fluktuasi biaya produksi dan jumlah permintaan secara presisi, sehingga perusahaan dapat melakukan pengendalian ketersediaan bahan baku dengan lebih efisien.

2.7 Apriori

Algoritma Apriori adalah salah satu algoritma dasar dalam *data mining* yang diusulkan oleh Agrawal dan Srikant pada tahun 1994, yang termasuk ke dalam

jenis *Association Rule Mining*. Algoritma ini berfungsi untuk menemukan aturan asosiatif atau mengungkapkan keterkaitan antar kombinasi item dalam suatu basis data transaksional, yang mana aturan tersebut sering juga dikenal dengan sebutan *Affinity Analysis* atau *Market Basket Analysis*. Dalam penerapannya, algoritma ini bekerja dengan berfokus pada analisis pola frekuensi tinggi (*frequent pattern mining*) melalui pencarian dan pengkombinasian kumpulan item yang paling sering muncul secara bersamaan (*frequent itemsets*). Proses pencarian ini sangat bergantung pada perhitungan dua tolok ukur yang utama, yaitu nilai *support* (persentase kemunculan suatu kombinasi item di dalam keseluruhan transaksi) dan nilai *confidence* (tingkat kepastian atau kuatnya hubungan antar item dalam aturan tersebut). Keunggulan dari algoritma Apriori adalah algoritmanya yang mudah untuk diimplementasikan, mampu memberikan hasil prediksi pola yang akurat, serta sangat tangguh untuk memproses sekumpulan data dalam jumlah yang sangat besar.

2.8 K-Means

K-Means merupakan salah satu algoritma clustering yang paling banyak digunakan dalam analisis data tak terstruktur (*unsupervised learning*). Algoritma ini bekerja dengan cara mengelompokkan sekumpulan data ke dalam k klaster berdasarkan kemiripan fitur, di mana setiap data *point* akan dimasukkan ke dalam klaster dengan *centroid* terdekat menggunakan ukuran jarak, umumnya *Euclidean distance*. Proses ini pertama kali diperkenalkan oleh MacQueen pada 1967 dan kemudian dikembangkan lebih lanjut oleh Hartigan & Wong pada 1979. K-Means tergolong dalam metode *partitional clustering* yang bersifat iteratif, di mana kualitas klaster diukur menggunakan fungsi objektif berupa minimisasi *within-cluster sum of squares* (WCSS) atau yang juga dikenal sebagai *inertia*. Algoritma ini banyak diterapkan dalam berbagai bidang seperti segmentasi pelanggan, pengenalan pola, kompresi citra, dan analisis dokumen teks.

2.9 Logistic Regression

Logistic Regression merupakan salah satu algoritma statistik dan machine learning yang paling luas digunakan untuk menyelesaikan permasalahan

klasifikasi, khususnya klasifikasi biner (dua kelas) yang termasuk ke dalam kategori supervised learning. Metode ini pertama kali dikembangkan secara mendalam oleh ahli statistika David Cox pada tahun 1958. Secara prinsip kerja, algoritma ini memodelkan probabilitas kejadian suatu kelas target berdasarkan satu atau lebih variabel independen (fitur masukan). Proses ini dilakukan dengan menghitung kombinasi linear dari fitur-fitur tersebut beserta parameter bobotnya, yang kemudian ditransformasikan menggunakan fungsi aktivasi logistik atau fungsi Sigmoid. Fungsi Sigmoid inilah yang berperan penting dalam memetakan keluaran regresi linear yang tidak terbatas menjadi nilai probabilitas dengan rentang pasti antara nol hingga satu. Keputusan akhir kemudian diambil dengan menetapkan nilai ambang batas (threshold), probabilitas yang melewati batas tersebut diklasifikasikan ke dalam kelas positif, dan sebaliknya ke dalam kelas negatif. Dalam pembelajarannya, kualitas pemodelan algoritma ini dioptimalkan dengan meminimalkan tingkat kesalahan pada Cost Function (seperti Log Loss) menggunakan metode Maximum Likelihood Estimation (MLE) atau Gradient Descent. Logistic Regression sangat digemari dan banyak diterapkan di industri karena modelnya yang efisien secara komputasi, hasil probabilitasnya yang mudah diinterpretasikan, serta sangat andal untuk berbagai kasus nyata seperti diagnosis penyakit, deteksi spam, hingga penilaian kelayakan risiko kredit.

BAB III

METODE PENELITIAN

Bab III Metode Penelitian ini menguraikan langkah-langkah logis dan rancangan komputasi algoritma untuk memecahkan setiap rumusan masalah. Pembahasan di dalamnya mencakup tahapan penemuan aturan asosiasi menggunakan algoritma Apriori (melalui ekstraksi item dan kalkulasi probabilitas *support* serta *confidence*), proses pengelompokan data tak berlabel dengan algoritma K-Means (meliputi penentuan kluster optimal via metode *Elbow* dan iterasi *centroid*), serta perancangan pemodelan klasifikasi biner menggunakan Logistic Regression (mencakup normalisasi data, aktivasi Sigmoid, evaluasi *Log Loss*, hingga optimasi *Gradient Descent* untuk prediksi akhir).

3.1 Algoritma Apriori

Metode pembentukan aturan asosiasi diawali dengan mengekstraksi seluruh data keranjang transaksi untuk mengidentifikasi dan mengumpulkan item-item unik tanpa duplikasi, sekaligus menghitung total populasi transaksi yang ada sebagai basis pembagi nilai. Tahap pembentukan kandidat dilakukan dengan menyusun seluruh kemungkinan kombinasi himpunan berukuran tertentu (seperti kombinasi dua atau tiga item) secara kombinatorial dari daftar item unik tersebut. Untuk mengetahui tingkat kepopuleran setiap kombinasi, metode pemindaian dilakukan dengan menghitung frekuensi kemunculan himpunan kombinasi di dalam keseluruhan data transaksi, yang kemudian dikonversi menjadi persentase nilai penunjang (*support*), di mana kombinasi yang persentasenya berada di bawah ambang batas minimum akan langsung dieliminasi. Sementara itu, penentuan tingkat kepastian hubungan sebab-akibat (*confidence*) menggunakan metode pemecahan kombinasi yang lolos seleksi menjadi himpunan prasyarat dan himpunan akibat secara iteratif berdasarkan panjang elemennya. Persentase *confidence* dihitung dengan membandingkan rasio kemunculan kombinasi item secara utuh terhadap frekuensi kemunculan item prasyaratnya saja di dalam basis data. Sebagai tahap akhir, aturan-aturan asosiasi yang berhasil melampaui standar minimum *confidence* dievaluasi lebih lanjut untuk mendapatkan nilai metrik final, yaitu dengan mengalikan persentase *support* dan *confidence* pembentuknya untuk memastikan tingkat keabsahan aturan tersebut.

3.2 K-Means

K-Means diimplementasikan melalui beberapa tahapan sistematis. Pertama, ditentukan nilai k (jumlah klaster) yang optimal menggunakan metode *Elbow*, yaitu dengan memplot nilai WCSS terhadap berbagai nilai k dan memilih titik di mana penurunan WCSS mulai melandai. Selanjutnya, algoritma diinisialisasi dengan menempatkan k *centroid* secara acak pada ruang fitur, kemudian setiap data *point* dihitung jaraknya terhadap seluruh *centroid* dan ditetapkan ke klaster dengan *centroid* terdekat. Setelah seluruh data dikelompokkan, posisi *centroid* diperbarui dengan menghitung rata-rata koordinat seluruh anggota klaster masing-masing. Proses penetapan anggota dan pembaruan *centroid* ini diulang secara iteratif hingga posisi *centroid* tidak lagi berubah secara signifikan atau jumlah iterasi maksimum tercapai. Evaluasi hasil *clustering* dilakukan menggunakan *Silhouette Score* untuk mengukur seberapa baik setiap data *point* cocok dengan klasternya dibandingkan klaster lain.

3.3 Logistic Regression

Logistic Regression diimplementasikan melalui serangkaian tahapan komputasi yang sistematis dan berurutan untuk melakukan klasifikasi biner. Alur matematis pembentukan model ini dijabarkan sebagai berikut.

3.3.1 Prapemrosesan dan Normalisasi Data (Z-Score Standardisation)

Sebelum data dimasukkan ke dalam model, variabel independen masukan (X) ditransformasikan menggunakan metode Z-Score untuk menyamakan skala antar-fitur. Proses ini bertujuan mencegah dominasi fitur berbobot nilai besar terhadap fitur lainnya, dengan persamaan berikut.

$$X_{norm} = \frac{X - \mu}{\sigma}$$

Keterangan

X_{norm} : Nilai terstandarisasi

X : Nilai data asli

μ : Rata-rata (mean) dari seluruh data

σ : Standar deviasi (simpangan baku) dari seluruh data

3.3.2 Perhitungan Kombinasi Linear (Forward Pass)

Sistem melakukan operasi perkalian dot matriks antara fitur input yang telah dinormalisasi (X_{norm}) dengan parameter bobot (weights atau w), kemudian menambahkannya dengan nilai penyeimbang atau bias (b). Proses ini menghasilkan skor mentah linear (logit atau z).

$$z = X_{norm} \times w + b$$

3.3.3 Transformasi Non-linear (Aktivasi Sigmoid)

Skor mentah linear (z) ditransformasikan ke dalam fungsi aktivasi logistik atau fungsi Sigmoid. Fungsi ini memetakan nilai logit yang tidak terbatas menjadi nilai probabilitas (y') dengan rentang pasti antara nol hingga satu.

$$y' = \frac{1}{1 + e^{-z}}$$

3.3.4 Evaluasi Tingkat Kesalahan (Cost Function)

Untuk mengukur seberapa jauh selisih antara nilai probabilitas hasil prediksi model (y') terhadap nilai target aktual yang sebenarnya (y), sistem mengevaluasinya secara global menggunakan fungsi biaya Log Loss atau Binary Cross-Entropy Loss.

$$J = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(y'^{(i)}) + (1 - y^{(i)}) \log(1 - y'^{(i)}) \right]$$

3.3.5 Perhitungan Turunan Parsial (Backward Pass)

Proses optimasi parameter diawali dengan menghitung nilai gradien atau turunan parsial dari fungsi biaya (J) terhadap bobot (dw) dan bias (db). Tahap ini mengukur arah dan besarnya sensitivitas perubahan parameter terhadap total tingkat kesalahan model.

a. Gradien Bobot (dw):

$$dw = \frac{1}{m} X_{norm}^T (y' - y)$$

b. Gradien Bias (db):

$$dw = \frac{1}{m} \sum_{i=1}^m (y'^{(i)} - y^{(i)})$$

Keterangan:

X_{norm}^T melambangkan matriks fitur input yang telah ditransposisi.

3.3.6 Pembaruan Parameter (Gradient Descent Update)

Nilai parameter bobot dan bias diperbarui secara simultan pada setiap iterasi dengan cara mengurangi nilai parameter lama menggunakan hasil kali antara laju pembelajaran (learning rate) dengan nilai gradien masing-masing parameter.

$$w = w - \alpha \times dw$$

$$b = b - \alpha \times db$$

Tahapan kedua hingga keenam ini diulangi secara berulang (looping) sepanjang jumlah epoch maksimum yang ditentukan atau hingga nilai fungsi biaya mencapai titik konvergensi (selisih error mendekati nol).

3.3.7 Pengambilan Keputusan (Thresholding) dan Evaluasi Akhir

Setelah model optimal didapatkan, proses prediksi tegas terhadap data uji biner ditentukan menggunakan nilai ambang batas (decision threshold) standar sebesar 0,5 dengan kondisi matematis.

$$\text{Prediksi Kelas} = \begin{cases} 1, & \text{jika } y' \geq 0.5 \\ 0, & \text{jika } y' < 0.5 \end{cases}$$

Kinerja pemodelan final dari algoritma Logistic Regression ini kemudian diukur dan divalidasi menggunakan metrik evaluasi confusion matrix untuk menghitung nilai tingkat akurasi, presisi, dan recall.

BAB IV

HASIL DAN PEMBAHASAN

Bab IV Hasil dan Pembahasan ini menyajikan implementasi kode program beserta visualisasi dan evaluasi dari setiap metode algoritma yang telah dirancang. Pembahasan di dalam bab ini mencakup pembuktian keberhasilan ekstraksi pola frekuensi tinggi dan aturan asosiasi (kombinasi *2-itemset* dan *3-itemset*) pada data keranjang belanja menggunakan algoritma Apriori, penentuan jumlah kluster optimal melalui metode *Elbow* serta visualisasi penyebaran kelompok data usia dan pendapatan pasien menggunakan K-Means, hingga hasil perhitungan komputasi probabilitas dan keakuratan model Logistic Regression dalam memprediksi risiko penyakit jantung secara medis melalui proses *forward propagation*, optimasi metrik *Log Loss*, dan *backward propagation*.

4.1 Algoritma Apriori

```
transaksi = [
    {"Susu", "Gula", "Teh"},
    {"Teh", "Gula", "Roti"},
    {"Teh", "Gula"},
    {"Susu", "Roti"},
    {"Susu", "Gula", "Roti"},
    {"Teh", "Gula"},
    {"Gula", "Kopi", "Susu"},
    {"Gula", "Kopi", "Susu"},
    {"Susu", "Roti", "Kopi"},
    {"Gula", "Teh", "Kopi"},
]

n_transaksi = len(transaksi)
n_transaksi

print(f"Jumlah transaksi: {n_transaksi}")

from itertools import combinations
semua_items = set()
for itemset in transaksi:
    semua_items.update(itemset)

print(f"Semua item unik: {semua_items}")
```

Kode Program 4.1 Kode Program Persiapan Data

Kode program di atas merupakan tahapan awal atau proses persiapan data dalam mengimplementasikan algoritma Apriori. Pertama, program mendefinisikan variabel transaksi yang berisi kumpulan simulasi data keranjang belanja pelanggan, di mana isi dari setiap keranjang tersebut direpresentasikan menggunakan tipe data

himpunan. Selanjutnya, program menghitung total keseluruhan jumlah transaksi menggunakan fungsi `len()` dan menyimpan angka hasilnya ke dalam variabel `n_transaksi` untuk ditampilkan ke layar. Setelah itu, program mengimpor fungsi `combinations` dari pustaka `itertools` yang nantinya akan digunakan untuk menyusun pasangan kombinasi barang. Langkah penting berikutnya adalah proses ekstraksi seluruh item program menyiapkan sebuah variabel himpunan kosong bernama `semua_items`, lalu melakukan perulangan untuk membaca setiap daftar belanja dan menggabungkan seluruh isinya menggunakan perintah `.update()`. Penggunaan tipe data set pada proses ini bertujuan untuk menyaring nama-nama barang secara otomatis agar menghasilkan daftar item yang unik tanpa adanya duplikasi, yang mana hasil akhir kumpulan barang unik ini kemudian dicetak ke layar sebagai bahan dasar untuk pencarian kombinasi pola frekuensi tinggi pada tahap selanjutnya.

4.1.1 F2

Dalam algoritma Apriori, kombinasi F2 atau yang biasa disebut sebagai 2-*itemset* merupakan sebuah himpunan yang berisi kombinasi tepat dua macam barang. Tujuan utama dari pembentukan kombinasi F2 ini adalah untuk mencari dan menganalisis pasangan dua barang apa saja yang paling sering dibeli secara bersamaan oleh pelanggan di dalam satu transaksi belanja. Pada proses kerjanya, kombinasi F2 baru akan dibentuk setelah algoritma selesai mendata barang-barang secara tunggal (1-*itemset*), yang kemudian setiap pasangan dua barang tersebut dihitung tingkat kemunculannya untuk mendapatkan persentase nilai *support*. Apabila persentase kemunculan dari pasangan barang F2 tersebut ternyata tidak memenuhi batas minimal *support* yang telah ditetapkan di awal, maka kombinasi barang tersebut akan langsung dihilangkan dan tidak diproses lebih lanjut untuk pembentukan aturan asosiasi.

4.1.1.1 Kode Program

```
f = 2
kombinasi_item = list(combinations(semua_items, f))
kombinasi_item

print(f"Kombinasi item dengan panjang {f}:")
kombinasi_item

min_support = 20
min_confidence = 35
```

```

print(f"Aturan asosiasi F{f} dengan minimum support
{min_support}% dan minimum confidence {min_confidence}%:")

for k in kombinasi_item:
    transaksi_count = 0
    for itemset in transaksi:
        if set(k).issubset(itemset):
            transaksi_count += 1
    persentase_support = (transaksi_count / n_transaksi) * 100
    if persentase_support < min_support:
        continue
    for i in range(1, len(k)):
        for a in combinations(k, i):
            b = tuple(set(k) - set(a))
            jumlah_a = 0
            for itemset in transaksi:
                if set(a).issubset(itemset):
                    jumlah_a += 1
            persentase_confidence = (transaksi_count / jumlah_a)
* 100
            if persentase_confidence < min_confidence:
                continue
            final = persentase_support * persentase_confidence /
100
            print(f"{a} -> {b}, Support:
{persentase_support:.2f}%, Confidence:
{persentase_confidence:.2f}%, Final: {final:.2f}%")

```

Kode Program 4.2 Kode Program Menghitung F2

Kode program di atas merupakan tahapan inti dalam pembentukan kombinasi dan pencarian aturan asosiasi pada algoritma Apriori. Pertama, program mendefinisikan variabel *f* dengan nilai 2 untuk menentukan panjang kombinasi barang, lalu merakit semua kemungkinan pasangan dari daftar item unik menggunakan fungsi `combinations()` dan menyimpannya ke dalam bentuk daftar pada variabel `kombinasi_item`. Selanjutnya, program menetapkan nilai ambang batas seleksi melalui variabel `min_support` sebesar 20 dan `min_confidence` sebesar 35. Setelah itu, program masuk ke dalam perulangan utama untuk mengevaluasi setiap pasangan barang dengan cara memindai seluruh transaksi dan menghitung frekuensi kemunculannya secara bersamaan menggunakan fungsi `issubset()`. Jika hasil kalkulasi `persentase_support` dari sebuah pasangan berada di bawah nilai batas minimal, program mengeksekusi perintah `continue` untuk langsung melewati pasangan tersebut. Langkah penting berikutnya, bagi kombinasi yang berhasil lolos uji minimum *support*, program melakukan perulangan bersarang untuk memecahnya menjadi skenario aturan sebab-akibat melalui operasi pengurangan himpunan `set(k) - set(a)`. Program kemudian

memindai ulang data transaksi untuk menghitung seberapa sering barang penyebab (a) dibeli secara mandiri, yang angkanya digunakan sebagai pembagi untuk menemukan nilai probabilitas `persentase_confidence`. Pada tahap akhir, aturan yang nilai *confidence*-nya gagal mencapai batas minimal akan dibuang, sedangkan aturan yang berhasil memenuhi kedua ambang batas tersebut akan dihitung nilai kalkulasi akhirnya pada variabel `final`, lalu dicetak ke layar beserta persentase masing-masing metrik evaluasinya sebagai hasil aturan asosiasi yang sah.

4.1.1.2 Hasil

```
Aturan asosiasi F2 dengan minimum support 20% dan minimum confidence 35%:
('Teh',) -> ('Gula',), Support: 50.00%, Confidence: 100.00%, Final: 50.00%
('Gula',) -> ('Teh',), Support: 50.00%, Confidence: 62.50%, Final: 31.25%
('Roti',) -> ('Susu',), Support: 30.00%, Confidence: 75.00%, Final: 22.50%
('Susu',) -> ('Roti',), Support: 30.00%, Confidence: 50.00%, Final: 15.00%
('Roti',) -> ('Gula',), Support: 20.00%, Confidence: 50.00%, Final: 10.00%
('Susu',) -> ('Gula',), Support: 40.00%, Confidence: 66.67%, Final: 26.67%
('Gula',) -> ('Susu',), Support: 40.00%, Confidence: 50.00%, Final: 20.00%
('Susu',) -> ('Kopi',), Support: 30.00%, Confidence: 50.00%, Final: 15.00%
('Kopi',) -> ('Susu',), Support: 30.00%, Confidence: 75.00%, Final: 22.50%
('Gula',) -> ('Kopi',), Support: 30.00%, Confidence: 37.50%, Final: 11.25%
('Kopi',) -> ('Gula',), Support: 30.00%, Confidence: 75.00%, Final: 22.50%
```

Gambar 4.1 Gambar Hasil Program Menghitung F2

Gambar di atas hasil dari program yang menampilkan daftar hasil akhir pencarian aturan asosiasi (*association rules*) dari sekumpulan data transaksi keranjang belanja. Pada program ini, algoritma Apriori memanipulasi pencarian pola kombinasi dua barang (F2) dengan menerapkan dua parameter ambang batas utama, yaitu memasukkan nilai minimum *support* sebesar 20% untuk memastikan pasangan barang cukup sering dibeli, dan minimum *confidence* sebesar 35% untuk memastikan tingkat kepastian hubungannya cukup kuat. Dari sisi pemrosesan, program ini mengevaluasi kekuatan setiap aturan berdasarkan perhitungan probabilitas bersyarat, di mana persentase *confidence* dihitung dengan membagi jumlah transaksi yang memuat kedua barang secara bersamaan dengan jumlah total transaksi dari barang penyebabnya (anteseden). Dengan metode penyaringan ini, program secara otomatis membuang seluruh pola yang lemah dan hanya mencetak aturan-aturan yang terbukti sah, seperti pada baris pertama yang memperlihatkan bahwa pelanggan yang membeli teh memiliki tingkat probabilitas mutlak (100%) untuk turut membeli gula, lengkap beserta skor nilai *Final* yang didapatkan dari hasil perkalian antara persentase *support* dan *confidence* pada setiap barisnya.

4.1.2 F3

Dalam algoritma Apriori, kombinasi F3 atau yang sering disebut sebagai *3-itemset* merupakan sebuah himpunan yang memuat kombinasi tepat tiga macam barang. Tujuan dari pembentukan kombinasi F3 ini adalah untuk mengidentifikasi pola pembelian yang lebih kompleks, di mana pelanggan membeli tiga barang sekaligus secara bersamaan di dalam satu transaksi, seperti contohnya kombinasi {Teh, Gula, Roti}. Secara prosedural, pembentukan dan evaluasi kombinasi F3 ini baru dilakukan setelah algoritma selesai memproses serta mendapatkan kombinasi barang yang valid pada tahap sebelumnya (yaitu *2-itemset* atau F2). Setiap kemungkinan pasangan tiga barang tersebut kemudian dievaluasi frekuensi kemunculannya secara bersama-sama di dalam seluruh basis data transaksi untuk mendapatkan persentase nilai penunjangnya (*support*). Apabila persentase kemunculan dari kombinasi tiga barang tersebut tidak mencapai batas minimum *support* yang telah ditentukan, maka kombinasi tersebut akan langsung digugurkan dan tidak akan diproses lebih lanjut untuk mencari nilai kepastian (*confidence*) atau membentuk aturan asosiasi.

4.1.2.1 Kode Program

```
f = 3
kombinasi_item = list(combinations(semua_items, f))
kombinasi_item

print(f"Kombinasi item dengan panjang {f}:")
kombinasi_item
min_support = 20
min_confidence = 35

print(f"Aturan asosiasi F{f} dengan minimum support
{min_support}% dan minimum confidence {min_confidence}%:")

for k in kombinasi_item:
    transaksi_count = 0
    for itemset in transaksi:
        if set(k).issubset(itemset):
            transaksi_count += 1
    persentase_support = (transaksi_count / n_transaksi) * 100
    if persentase_support < min_support:
        continue
    for i in range(1, len(k)):
        for a in combinations(k, i):
            b = tuple(set(k) - set(a))
            jumlah_a = 0
```

```

        for itemset in transaksi:
            if set(a).issubset(itemset):
                jumlah_a += 1
            persentase_confidence = (transaksi_count / jumlah_a)
* 100
            if persentase_confidence < min_confidence:
                continue
            final = persentase_support * persentase_confidence /
100
            print(f"{a} -> {b}, Support:
{persentase_support:.2f}%, Confidence:
{persentase_confidence:.2f}%, Final: {final:.2f}%")

```

Kode Program 4.3 Kode Program Menghitung F3

Kode program di atas merupakan tahapan lanjutan dalam pembentukan kombinasi dan pencarian aturan asosiasi pada algoritma Apriori untuk kelompok tiga barang. Pertama, program mendefinisikan variabel f dengan nilai 3 untuk menetapkan panjang kombinasi, lalu merakit semua kemungkinan himpunan tiga barang dari daftar item unik menggunakan fungsi `combinations()` dan menyimpannya ke dalam variabel `kombinasi_item`. Selanjutnya, program menetapkan nilai ambang batas seleksi melalui variabel `min_support` sebesar 20 dan `min_confidence` sebesar 35. Setelah itu, program masuk ke dalam perulangan utama untuk mengevaluasi setiap kombinasi tiga barang dengan cara memindai seluruh transaksi dan menghitung frekuensi kemunculannya secara bersamaan menggunakan fungsi `issubset()`. Jika hasil kalkulasi `persentase_support` dari suatu kombinasi berada di bawah ambang batas minimal, program mengeksekusi perintah `continue` untuk langsung melewati kombinasi tersebut. Langkah penting berikutnya, bagi kombinasi trio yang berhasil lolos uji minimum *support*, program melakukan perulangan bersarang yang dinamis menggunakan fungsi `range(1, len(k))` untuk memecahnya menjadi berbagai variasi skenario aturan sebab-akibat baik dari satu barang anteseden (a) ke dua barang konsekuen (b), maupun dari dua barang anteseden ke satu barang konsekuen melalui operasi pengurangan himpunan `set(k) - set(a)`. Program kemudian memindai ulang data transaksi untuk menghitung seberapa sering himpunan barang penyebab (a) dibeli oleh pelanggan, yang angkanya digunakan sebagai nilai pembagi untuk menemukan probabilitas `persentase_confidence`. Pada tahap akhir, variasi aturan yang nilai *confidence*-nya gagal mencapai batas minimal akan dibuang, sedangkan aturan yang berhasil memenuhi kedua ambang batas ketat tersebut akan dikalkulasi nilai akhirnya pada

variabel *final*, lalu dicetak ke layar beserta persentase masing-masing metrik evaluasinya sebagai hasil aturan asosiasi tingkat lanjut yang sah.

4.1.2.2 Hasil

```
Aturan asosiasi F3 dengan minimum support 20% dan minimum confidence 35%:  
( 'Kopi', ) -> ( 'Susu', 'Gula', ), Support: 20.00%, Confidence: 50.00%, Final: 10.00%  
( 'Susu', 'Gula', ) -> ( 'Kopi', ), Support: 20.00%, Confidence: 50.00%, Final: 10.00%  
( 'Susu', 'Kopi', ) -> ( 'Gula', ), Support: 20.00%, Confidence: 66.67%, Final: 13.33%  
( 'Gula', 'Kopi', ) -> ( 'Susu', ), Support: 20.00%, Confidence: 66.67%, Final: 13.33%
```

Gambar 4.2 Hasil Menghitung F3

Gambar di atas merupakan luaran (*output*) antarmuka terminal dari program yang menampilkan hasil akhir pencarian aturan asosiasi (*association rules*) tingkat lanjut untuk kelompok kombinasi tiga barang (F3). Pada tahap ini, algoritma Apriori menyeleksi pola himpunan tiga barang yang telah berhasil memenuhi parameter ambang batas minimum *support* sebesar 20% dan minimum *confidence* sebesar 35%. Berbeda dengan luaran pada tahap F2, daftar aturan yang dicetak ke layar kali ini memperlihatkan variasi skenario hubungan sebab-akibat yang lebih kompleks, mencakup aturan dari satu barang pemicu (*antecedent*) ke dua barang akibat (*konsekuen*), maupun dari kombinasi dua barang pemicu ke satu barang akibat. Dari sisi pemrosesan, program telah mengevaluasi probabilitas bersyarat dari pola-pola tersebut dan membuang kombinasi yang lemah, menyisakan aturan-aturan sah seperti yang ditunjukkan pada baris ketiga di mana pelanggan yang telah membeli kombinasi susu dan kopi secara bersamaan terbukti memiliki tingkat kepastian (*confidence*) yang cukup tinggi, yaitu sebesar 66,67%, untuk turut membeli gula. Seluruh aturan final tersebut ditampilkan secara rapi beserta metrik persentase *support* untuk mengukur tingkat frekuensi kombinasinya dalam total transaksi, serta metrik skor *Final* yang merepresentasikan nilai gabungan mutlak dari setiap aturan asosiasi yang dihasilkan.

4.2 K-Means

Data yang digunakan dalam percobaan ini yaitu data yang berisikan fitur *Age* dan *Income* sebanyak 10 sampel. Berikut merupakan data yang akan digunakan untuk percobaan *Clustering* menggunakan algoritma K-Means.

Tabel 4.1 Data *Clustering* menggunakan K-Means

No.	Age	Income
1	41	19
2	47	100
3	33	57
4	29	19
5	47	253
6	40	81
7	38	56
8	42	64
9	26	18
10	47	115

Data yang digunakan dalam penelitian ini terdiri dari 10 sampel dengan dua variabel, yaitu *Age* (usia) dan *Income* (pendapatan). Variabel usia berkisar antara 26 hingga 47 tahun, dengan beberapa sampel memiliki usia yang sama, mengindikasikan adanya distribusi yang tidak merata. Sementara itu, variabel pendapatan menunjukkan variasi yang cukup signifikan, mulai dari nilai terendah sebesar 18 hingga tertinggi 253, yang mengindikasikan adanya kesenjangan pendapatan yang cukup besar antar sampel. Pola sebaran data ini menjadikannya relevan untuk dianalisis menggunakan algoritma K-Means *Clustering*, guna mengidentifikasi pengelompokan alami berdasarkan kedekatan karakteristik usia dan pendapatan masing-masing individu.

4.2.1 Kode Program

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.cluster import KMeans
from yellowbrick.cluster import KElbowVisualizer
from sklearn.preprocessing import StandardScaler

data = {
    "Age": [41, 47, 33, 29, 47, 40, 38, 42, 26, 47],
    "Income": [19, 100, 57, 19, 253, 81, 56, 64, 18, 115]
}

df = pd.DataFrame(data)

standard_scaler = StandardScaler()
df_scaled = standard_scaler.fit_transform(df)
df_scaled = pd.DataFrame(df_scaled, columns=df.columns)
df_scaled

model = KMeans(random_state=42)
```

```
visualizer = KElbowVisualizer(  
    model,  
    k=(1, 11),  
    metric='distortion',  
    timings=False,  
)  
visualizer.fit(df_scaled)  
visualizer.show()  
  
k = visualizer.elbow_value_  
model = KMeans(n_clusters=k, random_state=42)  
model.fit(df_scaled)
```

```

df_results = df.copy()
df_results['Cluster'] = model.labels_
df_results

centroid_point_scaled = model.cluster_centers_
centroid_point =
standard_scaler.inverse_transform(centroid_point_scaled)
k = model.n_clusters

for i in range(k):
    print(f"Centroid {i}: {centroid_point[i]}")

plt.figure(figsize=(6,6))
plt.title(f"K = {k}")
sns.scatterplot(x=df_results.iloc[:, 0], y=df_results.iloc[:, 1],
hue=df_results['Cluster'], palette='Set2')
for i in range(len(df_results)):
    plt.text(df_results.iloc[i, 0], df_results.iloc[i, 1],
str(i), fontsize=9, ha='right')
for i in range(k):
    plt.text(centroid_point[i, 0], centroid_point[i, 1]+5,
f"C{i}", fontsize=9, ha='left', color='red')
plt.scatter(centroid_point[:, 0], centroid_point[:, 1],
color='black', s=50, marker='X', label='Centroid')

plt.show()

fig, axes = plt.subplots(4, 3, figsize=(12, 15))
axes = axes.flatten()

for idx, i in enumerate(range(1, 11)):
    model_iter = KMeans(n_clusters=i, random_state=42)
    model_iter.fit(df_scaled)

    k = model_iter.n_clusters

    df_results_iter = df.copy()
    df_results_iter['Cluster'] = model_iter.labels_

    ax = axes[idx]

    sns.scatterplot(
        x=df_results_iter.iloc[:, 0],
        y=df_results_iter.iloc[:, 1],
        hue=df_results_iter['Cluster'],
        palette='Set2',
        ax=ax,
        legend=False
    )

    centroids_scaled = model_iter.cluster_centers_
    centroids =
standard_scaler.inverse_transform(centroids_scaled)

    ax.scatter(
        centroids[:, 0],
        centroids[:, 1],
        color='black',
        s=80,

```

```

        marker='X',
        label='Centroid'
    )

    for j in range(len(df_results_iter)):
        ax.annotate(
            str(j),
            (df_results_iter.iloc[j, 0], df_results_iter.iloc[j,
1]),
            textcoords="offset points",
            xytext=(0, 8),
            ha='right',
            fontsize=8
        )

    for j in range(k):
        ax.annotate(
            f"C{j}",
            (centroids[j, 0], centroids[j, 1]),
            textcoords="offset points",
            xytext=(0, -15),
            ha='left',
            fontsize=9,
            color='red'
        )

    ax.set_title(f"K = {i}, Distortion =
{model_iter.inertia_.2f}")
    ax.set_xlabel('')
    ax.set_ylabel('')

for ax in axes[10:]:
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Kode Program 4.4 *Clustering menggunakan K-Means*

Kode diawali dengan mengimpor *library* yang dibutuhkan, yaitu *matplotlib* dan *seaborn* untuk visualisasi, *pandas* untuk manipulasi data, *sklearn* untuk algoritma K-Means dan normalisasi data, serta *yellowbrick* untuk visualisasi metode *Elbow*. Data yang terdiri dari variabel *Age* dan *Income* kemudian dimuat ke dalam *DataFrame*, lalu dinormalisasi menggunakan *StandardScaler* agar kedua variabel berada pada skala yang sebanding dan tidak ada variabel yang mendominasi proses *clustering* akibat perbedaan satuan atau rentang nilai.

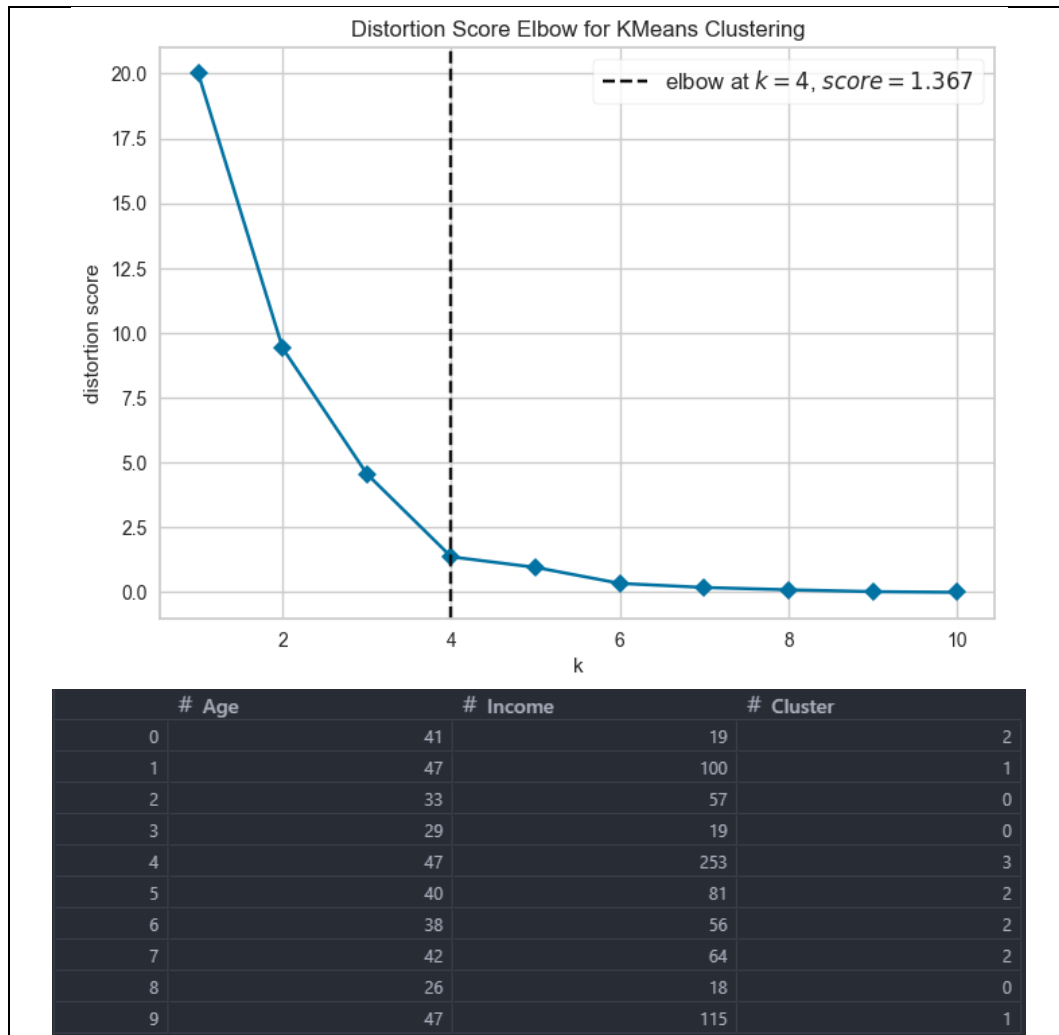
Setelah data dinormalisasi, kode menentukan jumlah kluster optimal menggunakan metode *Elbow* melalui *KElbowVisualizer* dari *library* *Yellowbrick*, dengan menguji nilai *k* dari 1 hingga 10 berdasarkan metrik *distortion* (WCSS). Nilai *k* optimal yang terdeteksi secara otomatis oleh *visualizer* kemudian digunakan untuk membangun model K-Means final. Model dilatih pada

data yang telah diskalakan, dan hasil label kluster ditambahkan ke `DataFrame` asli. *Centroid* yang dihasilkan kemudian dikembalikan ke skala aslinya menggunakan `inverse_transform` agar dapat diinterpretasikan secara nyata, lalu dicetak beserta koordinatnya.

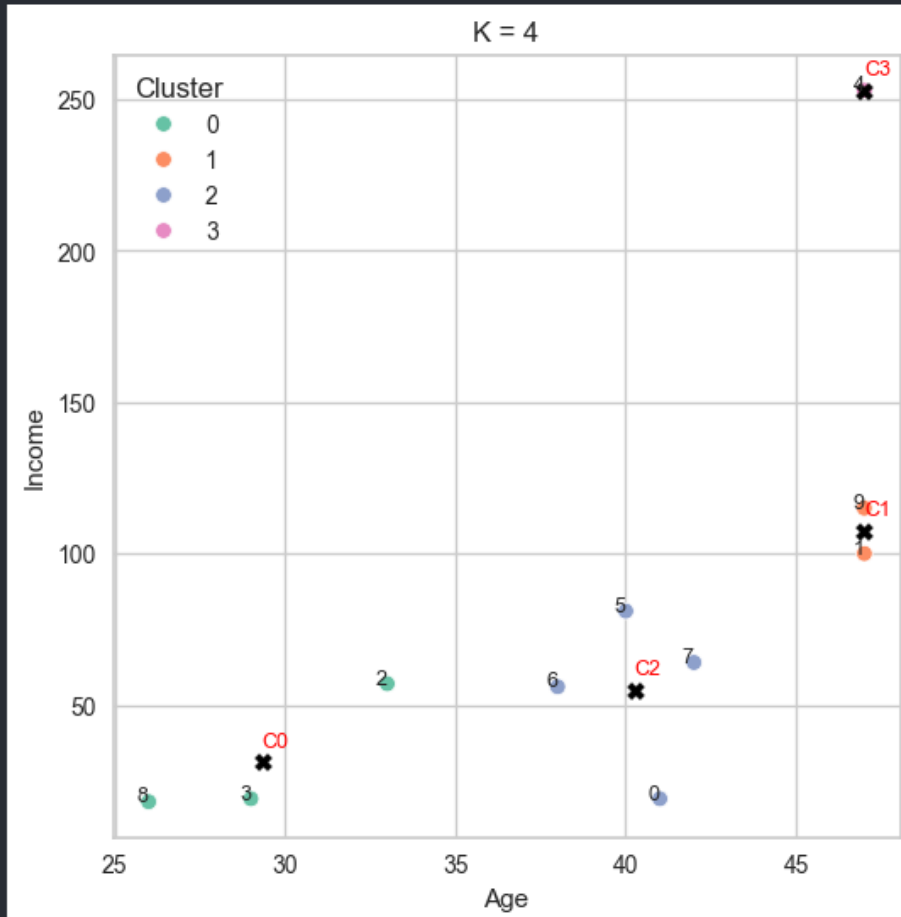
Pada tahap visualisasi, kode menghasilkan dua jenis plot. Pertama, *scatter* plot tunggal yang menampilkan hasil *clustering* optimal dengan label indeks tiap titik data dan posisi *centroid* yang ditandai dengan simbol silang berwarna hitam serta label "C0", "C1", dst. berwarna merah. Kedua, *subplot* berukuran 4×3 yang menampilkan perbandingan hasil *clustering* untuk setiap nilai k dari 1 hingga 10 secara bersamaan, masing-masing dilengkapi dengan nilai distorsi pada judulnya. Hal ini memudahkan analisis visual tentang bagaimana pembagian kluster berubah seiring bertambahnya nilai k, serta membantu memvalidasi pilihan k optimal yang diperoleh dari metode *Elbow* sebelumnya.

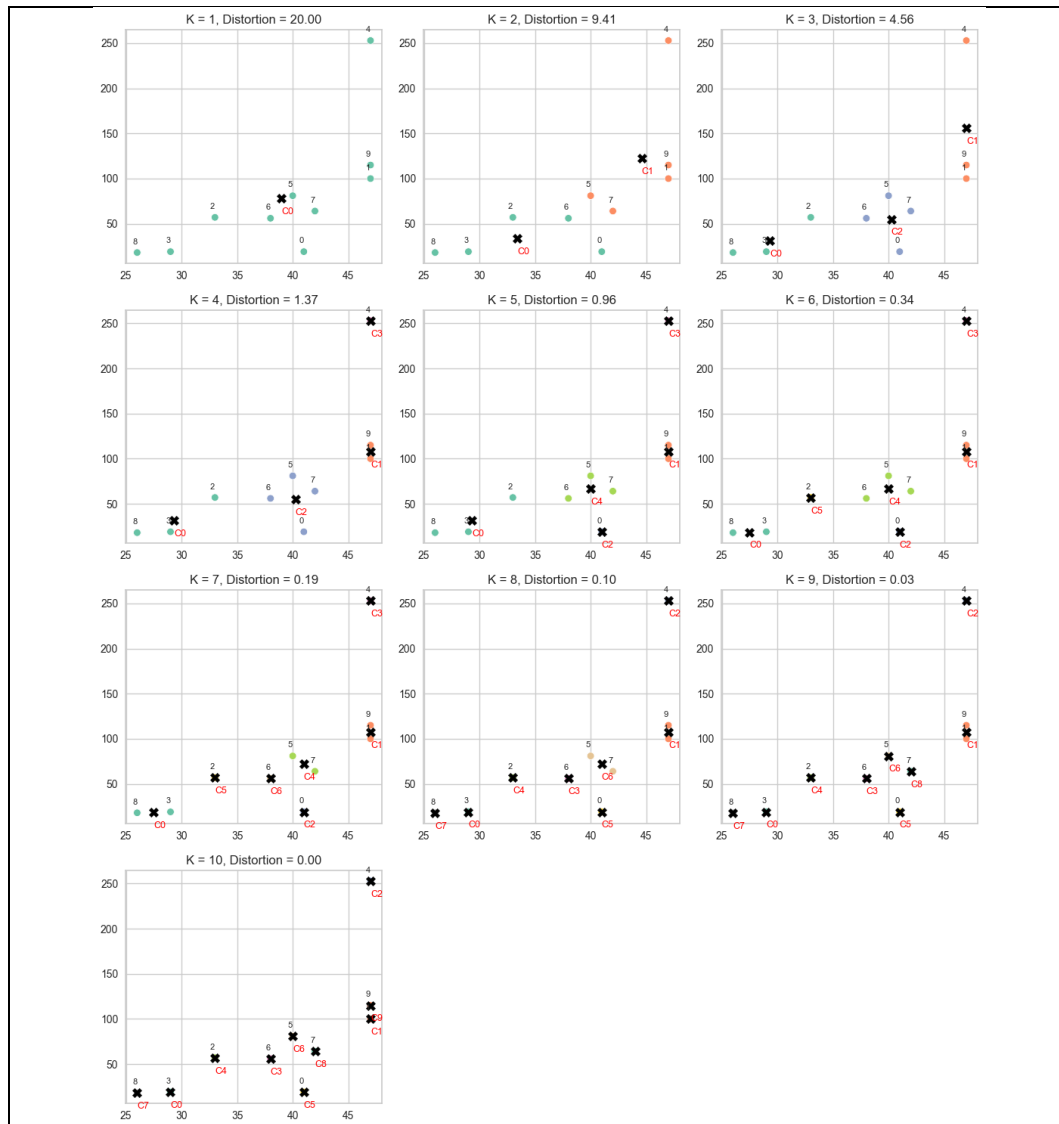
4.2.2 Hasil

	# Age	# Income
0	0.2795084971874737	-0.8907584965204536
1	1.118033988749895	0.328015797705167
2	-0.8385254915624212	-0.31898783997016245
3	-1.3975424859373686	-0.8907584965204536
4	1.118033988749895	2.6301450201313394
5	0.13975424859373686	0.04213046943002141
6	-0.13975424859373686	-0.33403443619517015
7	0.4192627457812106	-0.21366166639510883
8	-1.8168052317185792	-0.9058050927454613
9	1.118033988749895	0.5537147410802818



```
Centroid 0: [29.33333333 31.33333333]  
Centroid 1: [ 47.  107.5]  
Centroid 2: [40.25 55.  ]  
Centroid 3: [ 47. 253.]
```





Gambar 4.3 Hasil *Clustering* menggunakan K-Means

Gambar tersebut merupakan hasil *Clustering* menggunakan K-Means. Sebelum proses *clustering* dilakukan, seluruh data dinormalisasi menggunakan `StandardScaler` sehingga setiap variabel memiliki *mean* 0 dan standar deviasi 1. Hasil normalisasi menunjukkan bahwa nilai *Age* dan *Income* telah dikonversi ke dalam skala yang sebanding, misalnya sampel ke-4 yang memiliki *Income* tertinggi (253) terwakili dengan nilai *scaled* sebesar 2.63, sedangkan sampel dengan nilai rendah seperti sampel ke-8 memiliki nilai negatif pada kedua variabel. Proses ini penting agar variabel *Income* yang memiliki rentang jauh lebih besar tidak mendominasi perhitungan jarak dalam algoritma K-Means.

Hasil visualisasi *Elbow* menunjukkan bahwa *distortion score* mengalami penurunan yang sangat tajam dari K=1 (skor 20.0) hingga K=4 (skor 1.367),

kemudian penurunannya mulai melandai secara signifikan setelah $K=4$. Titik tekukan (*elbow*) terdeteksi secara otomatis oleh *KElbowVisualizer* pada $K=4$ dengan *distortion score* sebesar 1.367, sehingga nilai tersebut ditetapkan sebagai jumlah kluster optimal untuk model K-Means pada *dataset* ini.

Setelah model K-Means dijalankan dengan $K=4$, setiap sampel berhasil ditetapkan ke dalam salah satu dari empat kluster. Hasilnya menunjukkan bahwa Kluster 0 beranggotakan sampel 2, 3, dan 8 yang memiliki karakteristik usia muda dengan pendapatan rendah hingga menengah bawah. Kluster 1 terdiri dari sampel 1 dan 9 dengan usia tinggi (47 tahun) dan pendapatan menengah atas (100–115). Kluster 2 mencakup sampel 0, 5, 6, dan 7 yang memiliki usia menengah (38–42 tahun) dengan pendapatan bervariasi. Sementara Kluster 3 hanya beranggotakan sampel 4, yaitu individu dengan pendapatan tertinggi sebesar 253.

Posisi *centroid* akhir setelah dikembalikan ke skala asli adalah: *Centroid* 0 di koordinat [29.33, 31.33], *Centroid* 1 di [47.0, 107.5], *Centroid* 2 di [40.25, 55.0], dan *Centroid* 3 di [47.0, 253.0]. *Scatter plot* pada gambar yang sama memperlihatkan pemisahan keempat kluster secara visual, di mana Kluster 3 tampak terisolasi di sudut kanan atas karena pendapatannya yang jauh lebih tinggi dibanding sampel lain, sementara kluster lainnya terkonsentrasi pada area usia 26–47 tahun dengan pendapatan di bawah 120.

Subplot perbandingan dari $K=1$ hingga $K=10$ memperlihatkan bagaimana kualitas pengelompokan berevolusi seiring pertambahan jumlah kluster. Pada $K=1$ seluruh data menjadi satu kluster tunggal dengan distorsi tertinggi (20.00), sedangkan pada $K=10$ setiap titik data menjadi klasternya sendiri dengan distorsi 0.00 namun tidak lagi memiliki makna pengelompokan yang berarti. Perbandingan ini memperkuat keputusan pemilihan $K=4$ sebagai titik optimal, karena pada kondisi tersebut terbentuk pengelompokan yang cukup kompak dan bermakna secara interpretasi tanpa mengorbankan generalisasi model.

4.3 Logistic Regression

Data yang digunakan dalam percobaan ini yaitu data rekam medis kesehatan pasien sebanyak 20 sampel dengan 5 variabel independen (fitur) dan 1

variabel dependen (target). Berikut merupakan data yang akan digunakan untuk percobaan klasifikasi menggunakan algoritma Logistic Regression.

Tabel 4.2 Data Pasien

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Risiko Jantung
45	0	198	127	0	0
60	1	240	136	1	1
35	1	202	121	0	0
68	0	232	140	1	1
50	0	205	130	0	0
55	1	232	133	1	1
32	0	178	119	0	0
65	1	247	139	0	1
42	1	213	125	0	0
58	0	217	134	1	0
70	1	255	142	1	1
38	0	187	122	0	0
62	0	223	137	1	0
48	1	222	128	1	0
53	0	209	131	0	0
66	1	249	139	1	1
40	0	190	124	0	0
57	1	235	134	0	0
69	0	233	141	1	1
30	1	195	118	0	0

Data yang digunakan dalam penelitian ini terdiri dari 20 sampel dengan lima variabel independen (fitur prediktor) dan satu variabel dependen (kelas target). Variabel independen mencakup indikator kesehatan numerik seperti Umur yang berkisar antara 30 hingga 70 tahun, Kolesterol dengan rentang nilai 178 hingga 255, serta Tekanan Darah dari 118 hingga 142. Selain itu, terdapat variabel independen bertipe kategorikal biner, yaitu status Perokok (0 untuk tidak, 1 untuk ya) dan status Diabetes (0 untuk tidak, 1 untuk ya). Variabel dependen pada data ini adalah Risiko Jantung, yang juga bersifat biner (0 untuk risiko rendah, 1 untuk risiko tinggi). Pola kombinasi antara variabel numerik dan kategorikal ini menjadikannya sangat

relevan untuk dianalisis menggunakan algoritma Logistic Regression, guna mengidentifikasi sejauh mana fitur-fitur kesehatan tersebut memengaruhi probabilitas seorang pasien untuk diklasifikasikan ke dalam kategori risiko penyakit jantung.

4.3.1 Perhitungan Manual

Sebelum proses perhitungan dimulai pada Epoch ke-0, bobot (w) dan bias (b) diinisialisasi dengan nilai nol. Tingkat pembelajaran (*learning rate*) juga ditetapkan.

- a. Bobot awal (w): $[0, 0, 0, 0, 0]$
- b. Bias awal (b): 0
- c. Learning rate: 0.1

Selanjutnya, karena rentang nilai antar fitur berbeda secara signifikan (misalnya umur puluhan, sedangkan kolesterol ratusan), dilakukan standarisasi menggunakan Z-Score agar model tidak bias terhadap nilai yang besar.

Tabel 4.3 Hasil Standarisasi Data

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes
-0.5697	-1	-0.9268	-0.535	-0.9045
0.6254	1	1.0098	0.6688	1.1055
-1.3664	1	-0.7423	-1.3375	-0.9045
1.2628	-1	0.6409	1.2038	1.1055
-0.1713	-1	-0.604	-0.1338	-0.9045
0.2271	1	0.6409	0.2675	1.1055
-1.6055	-1	-1.8489	-1.605	-0.9045
1.0238	1	1.3325	1.07	-0.9045
-0.8087	1	-0.2351	-0.8025	-0.9045
0.4661	-1	-0.0507	0.4013	1.1055
1.4222	1	1.7014	1.4713	1.1055
-1.1274	-1	-1.4339	-1.2038	-0.9045
0.7848	-1	0.2259	0.8025	1.1055
-0.3307	1	0.1798	-0.4013	1.1055
0.0677	-1	-0.4196	0	-0.9045
1.1035	1	1.4247	1.07	1.1055
-0.9681	-1	-1.2956	-0.9363	-0.9045

0.3864	1	0.7792	0.4013	-0.9045
1.3425	-1	0.687	1.3375	1.1055
-1.7648	1	-1.0651	-1.7388	-0.9045

4.3.1.1 Proses Forward Propagation (Iterasi Pertama / Epoch 0)

Pada tahap ini, model menghitung skor mentah (Logit) lalu mengubahnya menjadi nilai probabilitas menggunakan fungsi aktivasi Sigmoid.

a. Menghitung Persamaan Linear

Pada setiap record data, sistem mengalikan nilai fitur yang telah dinormalisasi dengan bobot masing-masing, lalu ditambahkan bias. Karena pada Epoch 0 semua bobot (w) dan bias (b) bernilai 0, maka untuk seluruh data:

$$z = (x_1 \times 0) + (x_2 \times 0) + (x_3 \times 0) + (x_4 \times 0) + (x_5 \times 0) + 0$$

$$z = 0$$

Tabel 4.4 Hasil Nilai Persamaan Linear

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z
-0.5697	-1	-0.9268	-0.535	-0.9045	0
0.6254	1	1.0098	0.6688	1.1055	0
-1.3664	1	-0.7423	-1.3375	-0.9045	0
1.2628	-1	0.6409	1.2038	1.1055	0
-0.1713	-1	-0.604	-0.1338	-0.9045	0
0.2271	1	0.6409	0.2675	1.1055	0
-1.6055	-1	-1.8489	-1.605	-0.9045	0
1.0238	1	1.3325	1.07	-0.9045	0
-0.8087	1	-0.2351	-0.8025	-0.9045	0
0.4661	-1	-0.0507	0.4013	1.1055	0
1.4222	1	1.7014	1.4713	1.1055	0
-1.1274	-1	-1.4339	-1.2038	-0.9045	0
0.7848	-1	0.2259	0.8025	1.1055	0
-0.3307	1	0.1798	-0.4013	1.1055	0
0.0677	-1	-0.4196	0	-0.9045	0
1.1035	1	1.4247	1.07	1.1055	0
-0.9681	-1	-1.2956	-0.9363	-0.9045	0
0.3864	1	0.7792	0.4013	-0.9045	0
1.3425	-1	0.687	1.3375	1.1055	0

-1.7648	1	-1.0651	-1.7388	-0.9045	0
---------	---	---------	---------	---------	---

b. Menghitung Probabilitas (y')

Nilai persamaan linear (z) dimasukkan ke dalam fungsi aktivasi Sigmoid untuk mendapatkan probabilitas (rentang 0 hingga 1).

$$y' = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = 0,5$$

Berdasarkan hasil probabilitas dari fungsi aktivasi Sigmoid, sistem memprediksi peluang 50% untuk semua data masuk ke antara kelas 1 atau 0 pada iterasi pertama ini.

Tabel 4.5 Hasil Nilai Probabilitas

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z	Nilai y'
-0.5697	-1	-0.9268	-0.535	-0.9045	0	0,5
0.6254	1	1.0098	0.6688	1.1055	0	0,5
-1.3664	1	-0.7423	-1.3375	-0.9045	0	0,5
1.2628	-1	0.6409	1.2038	1.1055	0	0,5
-0.1713	-1	-0.604	-0.1338	-0.9045	0	0,5
0.2271	1	0.6409	0.2675	1.1055	0	0,5
-1.6055	-1	-1.8489	-1.605	-0.9045	0	0,5
1.0238	1	1.3325	1.07	-0.9045	0	0,5
-0.8087	1	-0.2351	-0.8025	-0.9045	0	0,5
0.4661	-1	-0.0507	0.4013	1.1055	0	0,5
1.4222	1	1.7014	1.4713	1.1055	0	0,5
-1.1274	-1	-1.4339	-1.2038	-0.9045	0	0,5
0.7848	-1	0.2259	0.8025	1.1055	0	0,5
-0.3307	1	0.1798	-0.4013	1.1055	0	0,5
0.0677	-1	-0.4196	0	-0.9045	0	0,5
1.1035	1	1.4247	1.07	1.1055	0	0,5
-0.9681	-1	-1.2956	-0.9363	-0.9045	0	0,5
0.3864	1	0.7792	0.4013	-0.9045	0	0,5
1.3425	-1	0.687	1.3375	1.1055	0	0,5
-1.7648	1	-1.0651	-1.7388	-0.9045	0	0,5

4.3.1.2 Menghitung Error / Cost Function (Log Loss)

Untuk mengetahui seberapa meleset tebakan awal tersebut, model menghitung Cost Function secara global untuk seluruh data latih ($m = 20$). Karena y' selalu 0.5 untuk semua baris pada Epoch 0, nilai $\log(0.5)$ bernilai konstan (sekitar -0.6931). Saat disubstitusikan terhadap nilai target y asli (0 atau 1), total rata-rata *error (cost)* awal adalah 0.6931.

4.3.1.3 Menghitung Gradien (Backward Propagation)

Untuk memperbaiki nilai bobot dan bias agar error mengecil, model menghitung nilai gradien dari Cost Function. Sistem menghitung selisih antara nilai prediksi dengan nilai aktual ($y' - y$), lalu dikalikan dengan fitur inputnya. Berdasarkan perhitungan selisih (dz) pada seluruh 20 baris data latih, diperoleh rata-rata gradien. Gradien inilah yang menjadi penunjuk arah seberapa besar parameter harus dikoreksi agar tingkat kesalahan menurun.

- a. Gradien bobot (dw): [-0.3504, -0.15, -0.3719, -0.3544, -0.2864]
- b. Gradien bias (db): 0.15

4.3.1.4 Pembaruan Parameter (Update Weight & Bias)

Setelah gradien (dw dan db) ditemukan, bobot dan bias diperbarui menggunakan laju pembelajaran (*learning rate*). Pada pemodelan ini ditetapkan nilai *learning rate* sebesar 0.1.

- a. Bobot baru (w): [0.035, 0.015, 0.0372, 0.0354, 0.0286]
- b. Bias baru (b): -0.015

Hasil dari proses pembaruan ini mengubah bobot dan bias yang sebelumnya bernilai 0 menjadi memiliki nilai bobot (w) dan bias (b) yang baru. Parameter baru inilah yang kemudian diserahkan untuk memulai iterasi pada Epoch 1.

4.3.1.5 Proses Forward Propagation (Iterasi Kedua / Epoch 1)

Pada tahap ini, model kembali menghitung skor mentah (Logit) menggunakan bobot dan bias terbaru lalu mengubahnya menjadi nilai probabilitas menggunakan fungsi aktivasi Sigmoid.

a. Menghitung Persamaan Linear

Pada setiap record data, sistem mengalikan nilai fitur yang telah dinormalisasi dengan bobot masing-masing, lalu ditambahkan bias. Karena pada Epoch 1 semua bobot (w) dan bias (b) sudah bukan bernilai 0 lagi, maka untuk seluruh data akan menghasilkan nilai yang bervariasi.

Tabel 4.6 Hasil Nilai Persamaan Linear

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z
-0.5697	-1	-0.9268	-0.535	-0.9045	-0.1293
0.6254	1	1.0098	0.6688	1.1055	0.1148
-1.3664	1	-0.7423	-1.3375	-0.9045	-0.1488
1.2628	-1	0.6409	1.2038	1.1055	0.1124
-0.1713	-1	-0.604	-0.1338	-0.9045	-0.0891
0.2271	1	0.6409	0.2675	1.1055	0.0729
-1.6055	-1	-1.8489	-1.605	-0.9045	-0.2378
1.0238	1	1.3325	1.07	-0.9045	0.0974
-0.8087	1	-0.2351	-0.8025	-0.9045	-0.0914
0.4661	-1	-0.0507	0.4013	1.1055	0.0303
1.4222	1	1.7014	1.4713	1.1055	0.1969
-1.1274	-1	-1.4339	-1.2038	-0.9045	-0.1914
0.7848	-1	0.2259	0.8025	1.1055	0.066
-0.3307	1	0.1798	-0.4013	1.1055	0.0125
0.0677	-1	-0.4196	0	-0.9045	-0.0691
1.1035	1	1.4247	1.07	1.1055	0.1612
-0.9681	-1	-1.2956	-0.9363	-0.9045	-0.1712
0.3864	1	0.7792	0.4013	-0.9045	0.0308
1.3425	-1	0.687	1.3375	1.1055	0.1217
-1.7648	1	-1.0651	-1.7388	-0.9045	-0.189

b. Menghitung Probabilitas (y')

Nilai persamaan linear (z) dimasukkan ke dalam fungsi aktivasi Sigmoid untuk mendapatkan probabilitas (rentang 0 hingga 1). Sesuai dengan nilai persamaan linear yang bervariasi, maka akan menghasilkan nilai probabilitas yang juga bervariasi untuk seluruh data.

Tabel 4.7 Hasil Nilai Probabilitas

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z	Nilai y'
-0.5697	-1	-0.9268	-0.535	-0.9045	-0.1293	0.4677
0.6254	1	1.0098	0.6688	1.1055	0.1148	0.5287
-1.3664	1	-0.7423	-1.3375	-0.9045	-0.1488	0.4629
1.2628	-1	0.6409	1.2038	1.1055	0.1124	0.5281
-0.1713	-1	-0.604	-0.1338	-0.9045	-0.0891	0.4777
0.2271	1	0.6409	0.2675	1.1055	0.0729	0.5182
-1.6055	-1	-1.8489	-1.605	-0.9045	-0.2378	0.4408
1.0238	1	1.3325	1.07	-0.9045	0.0974	0.5243
-0.8087	1	-0.2351	-0.8025	-0.9045	-0.0914	0.4772
0.4661	-1	-0.0507	0.4013	1.1055	0.0303	0.5076
1.4222	1	1.7014	1.4713	1.1055	0.1969	0.5491
-1.1274	-1	-1.4339	-1.2038	-0.9045	-0.1914	0.4523
0.7848	-1	0.2259	0.8025	1.1055	0.066	0.5165
-0.3307	1	0.1798	-0.4013	1.1055	0.0125	0.5031
0.0677	-1	-0.4196	0	-0.9045	-0.0691	0.4827
1.1035	1	1.4247	1.07	1.1055	0.1612	0.5402
-0.9681	-1	-1.2956	-0.9363	-0.9045	-0.1712	0.4573
0.3864	1	0.7792	0.4013	-0.9045	0.0308	0.5077
1.3425	-1	0.687	1.3375	1.1055	0.1217	0.5304
-1.7648	1	-1.0651	-1.7388	-0.9045	-0.189	0.4529

4.3.1.6 Menghitung Error / Cost Function (Log Loss)

Untuk mengetahui seberapa meleset tebakan awal tersebut, model menghitung Cost Function secara global untuk seluruh data latih ($m = 20$). Berbeda dengan Epoch 0 di mana nilai probabilitas y' bersikap seragam sebesar 0,5, pada Epoch 1 ini nilai probabilitas untuk setiap baris data sudah bervariasi karena dipengaruhi oleh karakteristik nilai fitur masing-masing data masukan. Berdasarkan hasil akumulasi seluruh nilai loss individu dari ke-20 baris data yang kemudian dibagi rata dengan jumlah sampel ($m = 20$), didapatkan total rata-rata *error (cost)* pada Epoch 1 mengalami penurunan menjadi 0.6439.

4.3.1.7 Menghitung Gradien (Backward Propagation)

Untuk memperbaiki nilai bobot dan bias agar error mengecil, model kembali menghitung nilai gradien dari Cost Function. Sistem menghitung selisih antara nilai prediksi dengan nilai aktual ($y' - y$), lalu dikalikan dengan fitur inputnya. Berdasarkan perhitungan selisih (dz) pada seluruh 20 baris data latih, diperoleh rata-rata gradien. Gradien inilah yang menjadi penunjuk arah seberapa besar parameter harus dikoreksi agar tingkat kesalahan menurun.

- a. Gradien bobot (dw): $[-0.3194, -0.1398, -0.3404, -0.3234, -0.2608]$
- b. Gradien bias (db): 0.1463

4.3.1.8 Pembaruan Parameter (Update Weight & Bias)

Setelah gradien (dw dan db) ditemukan, bobot dan bias diperbarui menggunakan laju pembelajaran (*learning rate*). Pada pemodelan ini ditetapkan nilai *learning rate* sebesar 0.1.

- a. Bobot baru (w): $[0.035, 0.015, 0.0372, 0.0354, 0.0286]$
- b. Bias baru (b): -0.015

Hasil dari proses pembaruan ini mengubah bobot dan bias menjadi memiliki nilai bobot (w) dan bias (b) yang baru. Parameter baru inilah yang kemudian diserahkan untuk memulai iterasi pada Epoch 2.

4.3.1.9 Proses Forward Propagation (Iterasi Ketiga / Epoch 2)

Pada tahap ini, model kembali menghitung skor mentah (Logit) menggunakan bobot dan bias terbaru lalu mengubahnya menjadi nilai probabilitas menggunakan fungsi aktivasi Sigmoid.

- a. Menghitung Persamaan Linear

Pada setiap record data, sistem mengalikan nilai fitur yang telah dinormalisasi dengan bobot masing-masing, lalu ditambahkan bias. Karena pada Epoch 2 semua bobot (w) dan bias (b) sudah bervariasi, maka untuk seluruh data akan kembali menghasilkan nilai yang bervariasi.

Tabel 4.8 Hasil Nilai Persamaan Linear

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z

-0.5697	-1	-0.9268	-0.535	-0.9045	-0.2485
0.6254	1	1.0098	0.6688	1.1055	0.219
-1.3664	1	-0.7423	-1.3375	-0.9045	-0.2852
1.2628	-1	0.6409	1.2038	1.1055	0.2137
-0.1713	-1	-0.604	-0.1338	-0.9045	-0.1717
0.2271	1	0.6409	0.2675	1.1055	0.1388
-1.6055	-1	-1.8489	-1.605	-0.9045	-0.4561
1.0238	1	1.3325	1.07	-0.9045	0.1859
-0.8087	1	-0.2351	-0.8025	-0.9045	-0.1754
0.4661	-1	-0.0507	0.4013	1.1055	0.0567
1.4222	1	1.7014	1.4713	1.1055	0.376
-1.1274	-1	-1.4339	-1.2038	-0.9045	-0.3673
0.7848	-1	0.2259	0.8025	1.1055	0.1249
-0.3307	1	0.1798	-0.4013	1.1055	0.0233
0.0677	-1	-0.4196	0	-0.9045	-0.1335
1.1035	1	1.4247	1.07	1.1055	0.3078
-0.9681	-1	-1.2956	-0.9363	-0.9045	-0.3287
0.3864	1	0.7792	0.4013	-0.9045	0.0584
1.3425	-1	0.687	1.3375	1.1055	0.2314
-1.7648	1	-1.0651	-1.7388	-0.9045	-0.3621

b. Menghitung Probabilitas (y')

Nilai persamaan linear (z) dimasukkan ke dalam fungsi aktivasi Sigmoid untuk mendapatkan probabilitas (rentang 0 hingga 1). Sesuai dengan nilai persamaan linear yang bervariasi, maka akan menghasilkan nilai probabilitas yang juga bervariasi untuk seluruh data.

Tabel 4.9 Hasil Nilai Probabilitas

Umur	Perokok	Kolesterol	Tekanan Darah	Diabetes	Nilai z	Nilai y'
-0.5697	-1	-0.9268	-0.535	-0.9045	-0.2485	0.4382
0.6254	1	1.0098	0.6688	1.1055	0.219	0.5545
-1.3664	1	-0.7423	-1.3375	-0.9045	-0.2852	0.4292
1.2628	-1	0.6409	1.2038	1.1055	0.2137	0.5532
-0.1713	-1	-0.604	-0.1338	-0.9045	-0.1717	0.4572
0.2271	1	0.6409	0.2675	1.1055	0.1388	0.5347
-1.6055	-1	-1.8489	-1.605	-0.9045	-0.4561	0.3879

1.0238	1	1.3325	1.07	-0.9045	0.1859	0.5463
-0.8087	1	-0.2351	-0.8025	-0.9045	-0.1754	0.4563
0.4661	-1	-0.0507	0.4013	1.1055	0.0567	0.5142
1.4222	1	1.7014	1.4713	1.1055	0.376	0.5929
-1.1274	-1	-1.4339	-1.2038	-0.9045	-0.3673	0.4092
0.7848	-1	0.2259	0.8025	1.1055	0.1249	0.5312
-0.3307	1	0.1798	-0.4013	1.1055	0.0233	0.5058
0.0677	-1	-0.4196	0	-0.9045	-0.1335	0.4667
1.1035	1	1.4247	1.07	1.1055	0.3078	0.5763
-0.9681	-1	-1.2956	-0.9363	-0.9045	-0.3287	0.4186
0.3864	1	0.7792	0.4013	-0.9045	0.0584	0.5146
1.3425	-1	0.687	1.3375	1.1055	0.2314	0.5576
-1.7648	1	-1.0651	-1.7388	-0.9045	-0.3621	0.4105

4.3.1.10 Menghitung Error / Cost Function (Log Loss)

Untuk mengetahui seberapa meleset tebakan awal tersebut, model menghitung Cost Function secara global untuk seluruh data latih ($m = 20$). Proses evaluasi tingkat kesalahan global pada Epoch 2 dilakukan menggunakan parameter bobot dan bias yang baru saja diperbarui untuk kedua kalinya. Karena arah pembaruan parameter yang dipandu oleh algoritma Gradient Descent bekerja secara optimal, nilai probabilitas prediksi y' pada setiap *record* data secara bertahap bergerak semakin mengerucut mendekati nilai target aktualnya (y). Keberhasilan optimasi ini ditunjukkan dengan penurunan nilai Cost Function yang berkelanjutan, di mana total rata-rata *error (cost)* secara global pada Epoch 2 berhasil ditekan hingga menyusut ke angka 0.6025.

4.3.1.11 Menghitung Gradien (Backward Propagation)

Untuk memperbaiki nilai bobot dan bias agar error mengecil, model kembali menghitung nilai gradien dari Cost Function. Sistem menghitung selisih antara nilai prediksi dengan nilai aktual ($y' - y$), lalu dikalikan dengan fitur inputnya. Berdasarkan perhitungan selisih (dz) pada seluruh 20 baris data latih, diperoleh rata-rata gradien. Gradien inilah yang menjadi penunjuk arah seberapa besar parameter harus dikoreksi agar tingkat kesalahan menurun.

- a. Gradien bobot (dw): [-0.2915, -0.1306, -0.312, -0.2955, -0.2376]

- b. Gradien bias (db): 0.1427

4.3.1.12 Pembaruan Parameter (Update Weight & Bias)

Setelah gradien (dw dan db) ditemukan, bobot dan bias diperbarui menggunakan laju pembelajaran (*learning rate*). Pada pemodelan ini ditetapkan nilai *learning rate* sebesar 0.1.

- a. Bobot baru (w): [0.0961, 0.042, 0.1024, 0.0973, 0.0785]
- b. Bias baru (b): -0.0439

Hasil dari proses pembaruan ini mengubah bobot dan bias menjadi memiliki nilai bobot (w) dan bias (b) yang baru. Karena ini merupakan iterasi terakhir dari simulasi perhitungan manual ini, maka bobot dan bias ini menjadi parameter akhir yang digunakan dalam model Logistic Regression untuk melakukan pengujian pada dataset uji yang ada.

4.3.2 Kode Program

```
import numpy as np
from sklearn.model_selection import train_test_split

data_pasien = {
    "umur": [45, 60, 35, 68, 50, 55, 32, 65, 42, 58, 70, 38, 62,
48, 53, 66, 40, 57, 69, 30],
    "perokok": [0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1,
0, 1, 0, 1],
    "kolesterol": [198, 240, 202, 232, 205, 232, 178, 247, 213,
217, 255, 187, 223, 222, 209, 249, 190, 235, 233, 195],
    "tekanan_darah": [127, 136, 121, 140, 130, 133, 119, 139,
125, 134, 142, 122, 137, 128, 131, 139, 124, 134, 141, 118],
    "diabetes": [0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1,
0, 0, 1, 0],
    "risiko_jantung": [0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0,
0, 1, 0, 0, 1, 0]
}

X = np.column_stack((
    data_pasien["umur"],
    data_pasien["perokok"],
    data_pasien["kolesterol"],
    data_pasien["tekanan_darah"],
    data_pasien["diabetes"]
))

y = np.array(data_pasien["risiko_jantung"])

print("Ukuran Data X:", X.shape)
print("Ukuran Data y:", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
)

print("\nUkuran X_train:", X_train.shape)
print("Ukuran X_test :", X_test.shape)

print("Ukuran y_train:", y_train.shape)
print("Ukuran y_test :", y_test.shape)

X_rata_rata = np.mean(X_train, axis=0)
X_std = np.std(X_train, axis=0)

X_train_norm = (X_train - X_rata_rata) / X_std
X_test_norm = (X_test - X_rata_rata) / X_std

print("\nContoh sebelum normalisasi:")
print(X_train[0])

print("\nContoh setelah normalisasi:")
print(X_train_norm[0])
```

Kode Program 4.5 *Persiapan Dataset*

Kode program di atas merupakan tahapan inialisasi dan prapemrosesan data (*data preprocessing*) sebelum model klasifikasi dibangun. Proses diawali dengan memuat *library* komputasi numerik `numpy` dan modul pembagian data `train_test_split` dari *library* `scikit-learn`. Dataset rekam medis pasien yang berisikan 20 sampel diinisialisasi menggunakan struktur data *dictionary*. Dari himpunan data tersebut, kelima variabel prediktor (umur, perokok, kolesterol, tekanan darah, dan diabetes) digabungkan secara vertikal menjadi sebuah matriks fitur dua dimensi berlabel X menggunakan fungsi `np.column_stack`. Sementara itu, variabel kelas target yaitu risiko jantung diekstrak menjadi *array* satu dimensi berlabel y.

Setelah matriks fitur dan target berhasil dibentuk, proses selanjutnya adalah mempartisi data menjadi himpunan data latih (*training set*) dan data uji (*testing set*) menggunakan rasio 80:20 (direpresentasikan oleh parameter `test_size=0.2`). Data latih (`x_train` dan `y_train`) yang terdiri dari 16 sampel akan dialokasikan untuk melatih model, sedangkan data uji (`x_test` dan `y_test`) yang terdiri dari 4 sampel disimpan secara terpisah untuk tahap pengujian akhir. Parameter `random_state=42` ditetapkan untuk mengunci pengacakan data sehingga hasil partisi akan selalu konsisten dan dapat direproduksi (*reproducible*) pada setiap eksekusi program.

Tahapan krusial terakhir pada blok kode ini adalah standardisasi data menggunakan metode Z-Score Standardization. Standardisasi ini bertujuan untuk menyeimbangkan rentang skala dari berbagai fitur (misalnya, perbedaan skala antara umur dalam rentang puluhan dengan kolesterol dalam rentang ratusan) agar tidak ada variabel yang mendominasi perhitungan komputasi secara bias. Model menghitung nilai rata-rata (`x_rata_rata`) dan standar deviasi (`x_std`) secara eksklusif hanya dari himpunan data latih untuk mencegah terjadinya kebocoran informasi (*data leakage*). Nilai statistik tersebut kemudian digunakan untuk mentransformasikan skala dari `x_train` menjadi `x_train_norm` dan `x_test` menjadi `x_test_norm`. Pada akhir eksekusi, sistem mencetak perbandingan matriks fitur baris pertama sebelum dan sesudah normalisasi untuk memastikan bahwa transformasi telah berhasil diterapkan dengan benar.

```
def hitung_z(X, w, b):
    return np.dot(X, w) + b
```

Kode Program 4.6 Rumus 1: Persamaan Linear

Kode program di atas mendefinisikan fungsi `hitung_z` yang berfungsi untuk melakukan komputasi persamaan linear atau menghitung nilai logit (z) pada tahap *forward propagation*. Fungsi ini menerima tiga parameter utama, yaitu x yang merepresentasikan matriks fitur *input* hasil normalisasi, w sebagai *array* parameter bobot (*weights*), serta b yang bertindak sebagai nilai bias. Di dalam fungsi, operasi perkalian titik matriks (*matrix dot product*) antara x dan w dieksekusi memanfaatkan fungsi `np.dot` dari *library* NumPy, lalu hasilnya ditambahkan dengan konstanta bias b . Luaran yang dihasilkan oleh fungsi ini berupa nilai skor mentah linear untuk setiap sampel data latih, yang selanjutnya akan diteruskan sebagai *input* bagi fungsi aktivasi Sigmoid untuk ditransformasikan menjadi nilai persentase probabilitas.

```
def hitung_sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

Kode Program 4.7 Rumus 2: Aktivasi Sigmoid

Kode program di atas mendefinisikan fungsi `hitung_sigmoid` yang bertugas menjalankan proses aktivasi non-linear dalam arsitektur algoritma Logistic Regression. Fungsi ini menerima parameter tunggal z , yang mewakili skor mentah linear (logit) dari hasil komputasi *forward pass* pada tahap sebelumnya. Secara operasional, fungsi ini merepresentasikan persamaan matematis fungsi logistik standar (Sigmoid), di mana komputasinya memanfaatkan modul `np.exp(-z)` dari *library* NumPy untuk menghitung nilai eksponensial negatif dari logit tersebut. Tujuan esensial dari transformasi ini adalah untuk memetakan hasil perhitungan kombinasi linear yang rentang nilainya tidak terbatas (*unbounded*) menjadi sebuah ukuran probabilitas yang valid. Melalui fungsi aktivasi ini, sistem dapat memastikan bahwa setiap nilai luaran prediksi yang dikembalikan akan selalu berada secara presisi pada rentang pecahan antara nol hingga satu.

```
def hitung_cost(y_asli, y_prediksi):
    m = len(y_asli)
    epsilon = 1e-15 # Mencegah error log(0)
    y_prediksi = np.clip(y_prediksi, epsilon, 1 - epsilon)
    cost = -(1/m) * np.sum(y_asli * np.log(y_prediksi) + (1 -
y_asli) * np.log(1 - y_prediksi))
    return cost
```

Kode Program 4.8 Rumus 3: Cost Function (Log Loss)

Kode program di atas mendefinisikan fungsi `hitung_cost` yang berperan krusial dalam mengevaluasi seberapa meleset tebakan model secara keseluruhan menggunakan metrik evaluasi Log Loss atau Binary Cross-Entropy. Fungsi ini menerima dua parameter, yaitu `y_asli` yang berisi *array* nilai kelas target aktual, dan `y_prediksi` yang merupakan *array* probabilitas hasil keluaran dari fungsi aktivasi Sigmoid sebelumnya. Di dalam fungsi, sistem terlebih dahulu menghitung total jumlah data yang diproses (m).

Tahapan penting pada fungsi ini adalah implementasi mekanisme pengamanan komputasi menggunakan variabel konstan epsilon yang bernilai sangat kecil ($1e-15$) yang dikombinasikan dengan metode pembatasan rentang dari NumPy, yaitu `np.clip`. Teknik pemotongan (*clipping*) ini secara paksa membatasi nilai probabilitas agar tidak pernah bernilai mutlak 0 atau 1. Hal ini wajib dilakukan untuk mencegah terjadinya error matematis berupa nilai tak terhingga (*infinity*) atau Not a Number (NaN) pada saat sistem mengeksekusi fungsi logaritma alami (`np.log`). Setelah nilai probabilitas aman dioperasikan, fungsi mengeksekusi operasi vektorisasi memanfaatkan `np.sum` untuk menghitung akumulasi kesalahan dari seluruh sampel data berdasarkan persamaan matematis fungsi biaya, sebelum akhirnya dibagi dengan total sampel (m) untuk mengembalikan satu nilai skalar tunggal yang merepresentasikan tingkat rata-rata *error (cost)* pada iterasi tersebut.

```
def hitung_gradien(X, y_asli, y_prediksi):
    m = len(y_asli)
    selisih = y_prediksi - y_asli
    dw = (1/m) * np.dot(X.T, selisih)
    db = (1/m) * np.sum(selisih)
    return dw, db
```

Kode Program 4.9 Rumus 4: *Gradient Descent*

Kode program di atas mendefinisikan fungsi `hitung_gradien` yang bertugas mengeksekusi tahapan komputasi *backward propagation* dalam arsitektur Logistic Regression. Fungsi ini menerima tiga parameter masukan, yakni matriks fitur latih (x), larik kelas target aktual (`y_asli`), dan larik probabilitas hasil tebakan model (`y_prediksi`). Proses kalkulasi diawali dengan menentukan jumlah total sampel data latih (m) dan menghitung vektor selisih atau *error* antara probabilitas tebakan model dengan target yang sesungguhnya. Selanjutnya, sistem mengkomputasi nilai turunan parsial fungsi biaya terhadap parameter bobot (dw).

Komputasi ini memanfaatkan operasi perkalian titik matriks (*dot product*) dari *library* NumPy antara matriks fitur yang telah ditransposisi ($X.T$) dengan vektor selisih tersebut, yang hasilnya kemudian dibagi dengan total sampel (m) untuk mendapatkan nilai rata-rata gradien. Secara bersamaan, komputasi turunan parsial terhadap bias (db) juga dilakukan dengan mengakumulasikan seluruh selisih menggunakan `np.sum` dan membaginya dengan jumlah sampel data. Pada akhir pengeksesekusiannya, fungsi ini mengembalikan matriks dw dan skalar db , yang secara matematis berfungsi sebagai penunjuk arah serta besaran koreksi yang diperlukan oleh sistem untuk memperbarui parameter bobot dan bias pada tahap iterasi selanjutnya.

```

jumlah_data, jumlah_fitur = X_train_norm.shape

w = np.zeros(jumlah_fitur)
b = 0.0

learning_rate = 0.1
epochs = 1000

history_cost = []

for i in range(epochs):

    z = hitung_z(X_train_norm, w, b)

    y_prediksi = hitung_sigmoid(z)

    cost = hitung_cost(y_train, y_prediksi)
    history_cost.append(cost)

    dw, db = hitung_gradien(X_train_norm, y_train, y_prediksi)

    w = w - (learning_rate * dw)
    b = b - (learning_rate * db)

    if i % 200 == 0:
        print(f"Epoch {i} | Error (Cost): {cost:.4f}")

print(f"\nTraining Selesai di Epoch {epochs}!")
print(f"Error Akhir: {history_cost[-1]:.4f}")

print("\n--- HASIL PEMBELAJARAN MODEL ---")
print("Bobot (w):", np.round(w, 4))
print("Bias (b):", round(b, 4))

print("--- HASIL PEMBELAJARAN MODEL ---")
print("Bobot (w):", np.round(w, 4))
print("Bias (b):", round(b, 4))

print("\n--- PERBANDINGAN PREDIKSI VS ASLI ---")

z_akhir = hitung_z(X_test_norm, w, b)

peluang_akhir = hitung_sigmoid(z_akhir)

prediksi_kelas = np.where(peluang_akhir >= 0.5, 1, 0)

prediksi_benar = prediksi_kelas == y_test

akurasi = np.mean(prediksi_benar) * 100

print(f>Data Asli : {y_test}")
print(f>Prediksi : {prediksi_kelas}")
print(f>Akurasi Model: {akurasi:.2f}%")

```

Kode Program 4.10 Proses *Training* dan Evaluasi Model

Kode program di atas merupakan tahapan inti dari proses pelatihan model (*training loop*) menggunakan mekanisme optimasi Gradient Descent. Sebelum iterasi dimulai, sistem menginisialisasi parameter bobot (w) dan bias (b) dengan

nilai nol, serta menetapkan *hyperparameter* pelatihan yang terdiri dari laju pembelajaran (*learning rate*) sebesar 0,1 dan jumlah iterasi maksimum (*epochs*) sebanyak 1.000 kali. Di dalam struktur perulangan `for`, sistem secara berurutan memanggil fungsi-fungsi komputasi dasar yang telah didefinisikan sebelumnya. Proses ini mencakup eksekusi *forward propagation* untuk menghasilkan nilai probabilitas (`y_prediksi`), evaluasi tingkat kesalahan model (*cost*), hingga perhitungan *backward propagation* untuk mendapatkan arah gradien (`dw` dan `db`). Berdasarkan nilai gradien tersebut, parameter bobot dan bias secara konstan diperbarui di setiap akhir iterasi guna meminimalkan *error*. Perkembangan metrik Cost Function ini juga disimpan ke dalam daftar `history_cost` dan dicetak setiap 200 iterasi untuk memvalidasi bahwa model benar-benar mengalami konvergensi menuju tingkat kesalahan yang paling minimum.

Setelah tahap pelatihan selesai dan parameter bobot serta bias paling optimal berhasil didapatkan, tahapan selanjutnya adalah melakukan pengujian (*testing*) dan evaluasi performa klasifikasi. Model melakukan *forward pass* satu kali lagi, namun kali ini menggunakan himpunan data uji (`x_test_norm`) yang sengaja disisihkan dari awal dan tidak ikut dilibatkan selama proses training. Dari proses ini, didapatkan probabilitas akhir (`peluang_akhir`) yang kemudian dikonversi menjadi keputusan tegas menggunakan fungsi ambang batas `np.where`. Probabilitas yang bernilai lebih besar atau sama dengan threshold 0,5 langsung diklasifikasikan sebagai kelas 1 (risiko jantung tinggi), sementara probabilitas di bawahnya dikategorikan sebagai kelas 0 (risiko jantung rendah). Sebagai tahap akhir penutup kode, sistem mencocokkan *array* tebakan model (`prediksi_kelas`) dengan *array* target aktual (`y_test`) untuk mengkalkulasi persentase akurasi akhir, sehingga keandalan model Logistic Regression dalam memprediksi data pasien yang belum pernah dilihat sebelumnya dapat terukur secara objektif.

4.3.3 Hasil

```
--- HASIL PEMBELAJARAN MODEL ---  
Bobot (w): [ 1.8519  1.6621  2.4524  1.9746 -0.5095]  
Bias (b): -2.9389  
  
--- PERBANDINGAN PREDIKSI VS ASLI ---  
Data Asli : [0 0 1 1]  
Prediksi  : [0 1 1 1]  
Akurasi Model: 75.00%
```

Gambar 4.4 Gambar Hasil Evaluasi Model

Gambar tersebut menunjukkan hasil evaluasi model klasifikasi risiko penyakit jantung yang dibuat menggunakan konsep regresi logistik sederhana dengan NumPy. Pada kode program, data pasien dibagi menjadi data latih dan data uji, kemudian fitur seperti umur, status perokok, kolesterol, tekanan darah, dan diabetes dinormalisasi sebelum dilatih menggunakan proses *gradient descent* selama 1000 *epoch*. Nilai bobot (w) menunjukkan pengaruh masing-masing fitur terhadap prediksi model, di mana bobot positif seperti umur, perokok, kolesterol, dan tekanan darah cenderung meningkatkan peluang pasien diprediksi berisiko jantung, sedangkan bobot negatif pada diabetes menunjukkan pengaruh penurunan terhadap hasil prediksi berdasarkan pola data latih yang digunakan. Nilai bias (b) sebesar -2.9389 berfungsi sebagai nilai dasar penyesuaian model. Pada tahap evaluasi, model membandingkan data asli [0 0 1 1] dengan hasil prediksi [0 1 1 1], sehingga terdapat 3 prediksi yang benar dari total 4 data uji. Oleh karena itu, akurasi model yang diperoleh adalah 75%, yang berarti model sudah cukup mampu mengenali pola risiko jantung, tetapi masih terdapat satu kesalahan prediksi pada data uji.

BAB V

PENUTUP

Bab V Penutup ini merupakan bagian penutup dari laporan yang memuat kesimpulan secara komprehensif mengenai hasil implementasi algoritma *data mining* dan *machine learning* (Apriori, K-Means, dan Logistic Regression) yang telah dilakukan. Selain itu, bab ini juga menyajikan saran untuk perbaikan dan pengembangan sistem di masa mendatang guna mengatasi berbagai batasan masalah yang ada.

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pembahasan dari algoritma penambangan data (*data mining*) yang telah dilakukan, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Pencarian Pola Asosiasi (Apriori): Sistem terbukti mampu mengekstrak pola kombinasi barang keranjang belanja melalui pemrosesan 2-itemset (F2) dan 3-itemset (F3) secara akurat. Dengan menerapkan ambang batas minimum support 20% dan minimum confidence 35%, algoritma ini secara efektif mengeliminasi pola yang lemah dan membentuk aturan sebab-akibat yang sah, seperti ditemukannya kepastian mutlak (100%) pada pembelian teh yang memicu pembelian gula, serta probabilitas 66,67% pada kombinasi pembelian susu dan kopi yang memicu pembelian gula.
2. Pengelompokan Data (K-Means): Pendekatan klustering dengan algoritma K-Means berhasil menyegmentasi sebaran data pasien tanpa label berdasarkan kedekatan karakteristik variabel usia (Age) dan pendapatan (Income). Melalui prapemrosesan standardisasi StandardScaler dan pemanfaatan visualisasi metode Elbow, sistem dengan tepat mendeteksi penurunan distorsi secara optimal pada nilai $K=4$, yang secara visual berhasil membagi populasi ke dalam 4 kelompok berkarakteristik unik.
3. Klasifikasi Prediktif Klinis (Logistic Regression): Pemodelan regresi logistik berhasil diimplementasikan secara komputasional untuk memprediksi probabilitas risiko penyakit jantung biner. Proses normalisasi Z-Score, kalkulasi forward propagation dengan fungsi aktivasi Sigmoid,

hingga pembaruan parameter melalui backward propagation (Gradient Descent) selama 1000 iterasi secara efektif mampu meminimalkan nilai error (Log Loss) secara bertahap. Pada evaluasi akhir, model terbukti mampu memetakan bobot setiap fitur medis dan mencapai akurasi sebesar 75% pada data uji.

5.2 Saran

Berdasarkan batasan masalah dan hasil penelitian, terdapat beberapa saran yang dapat dilakukan untuk pengembangan sistem di masa mendatang:

1. Peningkatan Skala Data Latih: Percobaan algoritma saat ini dibatasi pada penggunaan sampel data yang berskala sangat kecil, yakni kisaran 10 hingga 20 sampel data saja. Untuk pengembangan selanjutnya, disarankan agar sistem diuji menggunakan himpunan data (dataset) bervolume besar guna memvalidasi kemampuan algoritma terhadap big data agar pola asosiasi yang tergali lebih kaya dan model prediksi yang terbentuk terhindar dari bias yang berlebihan.
2. Optimasi Evaluasi Klustering: Penentuan validitas dan jumlah kluster optimal pada implementasi K-Means saat ini hanya bergantung pada metode Elbow berbasis metrik minimisasi distorsi (WCSS). Disarankan untuk mengintegrasikan metrik evaluasi tambahan seperti penghitungan Silhouette Score pada tahap akhir agar perbandingan kepadatan dan jarak keterpisahan antar-kluster dapat diukur secara lebih objektif dan kuantitatif.
3. Penyempurnaan Metode Klasifikasi: Akurasi pengujian model Logistic Regression yang saat ini berada pada angka 75% masih dapat ditingkatkan. Disarankan untuk menerapkan mekanisme pencarian parameter hibrida (Hyperparameter Tuning) untuk mendapatkan besaran nilai learning rate dan iterasi epoch yang paling ideal. Di samping itu, dapat diimplementasikan pula komparasi dengan algoritma klasifikasi kompleks lainnya, seperti Support Vector Machine (SVM) atau Random Forest, guna menghasilkan deteksi penyakit medis yang jauh lebih mutakhir dan akurat.

DAFTAR PUSTAKA

- Abidin, Z., Amartya, A. K., & Nurdin, A. (2022). PENERAPAN ALGORITMA APRIORI PADA PENJUALAN SUKU CADANG KENDARAAN RODA DUA (STUDI KASUS: TOKO PRIMA MOTOR SIDOMULYO). *JURNAL TEKNOINFO*, 225-232.
- ALASALI, T., & ORTAKCI, Y. (2024). Clustering Techniques in Data Mining: A Survey of Methods, Challenges, and Applications. *Journal of Computer Science*, 32-50.
- Amri, K., Nazir, A., Haerani, E., Affandes, M., Candra, R. M., & Akhyar, A. (2022). Penerapan Data Mining Dalam Mencari Pola Asosiasi Data Tracer Study Menggunakan Equivalence Class Transformation (ECLAT). *Jurnal Nasional Komputasi dan Teknologi Informasi*, 442-449.
- Anshori, M., & Haris, M. S. (2022). Predicting Heart Disease using Logistic Regression. *Knowledge Engineering and Data Science (KEDS)*, 188-196.
- Bao, W., Cao, Y., Yang, Y., Che, H., Huang, J., & Wen, S. (2024). Data-driven stock forecasting models based on neural networks: A review. *Information Fusion*, 1566-2535.
- Barbounakis, I. S. (2022). Data Mining Methods: A Review. *International Journal of Computer Applications*, 5-19.
- Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 100–108. Diambil kembali dari <https://doi.org/10.2307/2346830>
- Lubis, A. S., Tugiono, & Hafizah. (2022). Data Mining Estimasi Biaya Produksi Ikan Kembung Rebus Dengan Regresi Linier Berganda. *JURNAL SISTEM INFORMASI TGD*, 888-897.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 281–297.
- Mardi, Y. (2017). Data Mining : Klasifikasi Menggunakan Algoritma C4.5. *Jurnal Edik Informatika*, 213-219.

- Nuaimi, H. S., Acquaye, A., & Mayyas, A. (2025). Machine learning applications for carbon emission estimation. *Resources, Conservation & Recycling Advances*, 2667-3789.
- Okolie, A., Obunadike, C., Okoro, S. C., Olufemi, I. B., Nwoke, P., & Akwabeng, P. M. (2025). Heart Disease Prediction: A Logistic Regression Approach. *Open Journal of Applied Sciences*, 3534-3552.
- Sahara, W., Saragih, S. D., & Windarto, A. P. (2022). Teknik Asosiasi Datamining Dalam Menentukan Pola Penjualan dengan Metode Apriori. *TIN: Terapan Informatika Nusantara*, 684-689.
- Shu, X., & Ye, Y. (2022). Knowledge Discovery: Methods from data mining and machine learning. *Social Science Research*, 1-16.
- Tarigan, P. M., Hardinata, J. T., Qurniawan, H., Safii, M., & Winanjaya, R. (2022). Implementasi Data Mining Menggunakan Algoritma Apriori Dalam Menentukan Persediaan Barang (Studi Kasus: Toko Sinar Harahap). *Jurnal Janitra Informatika dan Sistem Informasi*, 9-19.